

**ENHANCED ARTIFICIAL RABBIT OPTIMIZATION
ALGORITHM FOR SOLVING THE UNIVERSITY COURSE
TIMETABLING PROBLEM: A COMPARATIVE ANALYSIS
WITH EXISTING METAHEURISTIC**

BY

**OLUFUNSO DAVID OLUWADARASIMI
(22CG031924)**

**A PROJECT SUBMITTED TO THE DEPARTMENT
OF COMPUTER AND INFORMATION SCIENCES,
COLLEGE OF SCIENCE AND TECHNOLOGY,
COVENANT UNIVERSITY OTA, OGUN STATE.**

**IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE AWARD OF THE
BACHELOR OF SCIENCE (HONOURS) DEGREE IN
COMPUTER SCIENCE**

JUNE, 2026

CERTIFICATION

I hereby certify that this project was carried out by Olufunso, David Oluwadarasimi, in the Department of Computer and Information Sciences, College of Science and Technology, Covenant University, Ota, Ogun State, Nigeria, under my supervision.

Miss Ti-Jesu Tunde-Adeleke

Supervisor

Signature and Date

Prof. Atinuke A. Oni

Head of Department

Signature and Date

DEDICATION

I express my gratitude to God Almighty for His grace and guidance throughout this project, and to my family for their unwavering support.

ACKNOWLEDGEMENTS

I thank God Almighty for His strength, wisdom, and constant mercy throughout my degree program. My deepest gratitude to my supervisor, Miss Ti-Jesu Tunde-Adeleke, the HOD, and the faculty for their invaluable guidance. I am thankful to my family and friends for their endless love, sacrifices, and unwavering moral support.

TABLE OF CONTENTS

CONTENT	PAGES
COVER	i
CERTIFICATION	ii
DEDICATION	iii
ACKNOWLEDGEMENTS	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	ix
LIST OF TABLES	x
ABBREVIATIONS	xi
ABSTRACT	xii
CHAPTER ONE:INTRODUCTION	1
1.1. Background Information	1
1.2. Statement of the Problem	2
1.3. Aim and Objectives of the Study	2
1.4. Methodology	3
1.5. Significance of the Study	4
1.6. Scope of the study	5
1.7. Project Outline	5
CHAPTER TWO: LITERATURE REVIEW	6
2.1. Preamble	6
2.2. Review of Relevant Concepts	6
2.2.1. University Course Timetabling Problem (UCTP)	6
2.2.2. Hard Constraints and Soft Constraints	9

2.2.3.	Metaheuristics	10
2.2.4	Hybrid Metaheuristics	11
2.2.5.	Hyper-heuristics	13
2.2.6.	Artificial Rabbit Optimization (ARO)	13
2.2.7.	Benchmark Datasets and Metrics of Evaluation.	16
2.2.8.	Artificial Intelligence and Machine Learning in timetabling.	18
2.3.	Review of Related Methods	19
2.3.1.	Genetic Algorithm-Based Approaches	19
2.3.2.	Simulated Annealing and Tabu Search	20
2.3.3.	Swarm Intelligence and Nature-Based Algorithms.	21
2.3.4.	Hyper-heuristic Approaches	22
2.3.5.	Mathematical Programming and Matheuristics	23
2.3.6.	Emerging AI and Machine Learning Approaches	26
2.4.	Summary of Literature Review	26
CHAPTER THREE: RESEARCH METHODOLOGY		28
3.1.	Preamble	28
3.2.	System Overview	28
3.3.	Architectural Design	28
3.3.1.	The University Course Timetabling Problem	30
3.3.2.	Hard Constraints	31
3.3.3.	Soft Constraints (Minimize)	32
3.3.4.	Solution Encoding	32
3.3.5.	Key Assumptions	33

3.4.	Datasets	34
3.4.1.	Dataset Format and Parser logic	34
3.4.2.	Experimental Environment	35
3.4.3.	Evaluation Performance Metrics	36
3.5.	Enhanced Artificial Rabbit Optimization	37
3.5.1.	Kempe Chain Perturbation (exploration operator)	38
3.5.2.	Slot-Swap local Search (exploitation operator)	39
3.5.3.	Greedy repair (feasibility recovery)	39
3.5.4.	Adaptive Phase Control	40
3.5.5.	Large Instance Mode	40
3.6.	Statistical Comparison Framework	40
3.6.1.	Evaluation Metrics	40
3.6.2.	Hypothesis Testing Approach	41
3.6.3.	Effect Size Reporting	42
3.6.4.	Interpretation Framework	44
CHAPTER FOUR: IMPLEMENTATION, RESULTS AND DISCUSSION		45
4.1.	Preamble	45
4.2.	Implementation Overview	45
4.3.	Core Components	46
4.3.1.	Fitness Evaluator	46
4.3.2.	Discrete Neighbourhood Operators	48
4.3.3.	Algorithm Implementations	48
4.4.	Benchmarking Protocol	50

4.5.	Overall Algorithm Performance	52
4.6.	Per-Instance Results	54
4.7.	Statistical Evaluation	57
4.8.	Discussion	60
4.8.1.	Why SA Outperforms Enhanced ARO	60
4.8.2.	Why Enhanced ARO Outperforms Standard ARO	61
4.8.3.	Role of Domain-Specific Operators	62
4.8.4.	Budget Limitations and their Implications	63
4.8.5.	Implications for Algorithm Selection	64
CHAPTER FIVE: SUMMARY, LIMITATIONS AND CONCLUSION		65
5.1.	Summary	65
5.2.	Limitations	66
5.3.	Conclusion	67
REFERENCES		68

LIST OF FIGURES

FIGURES	TITLES OF FIGURES	PAGES
2.1	Flowchart of the heuristic algorithm steps for university course timetabling	8
2.2	Genetic Algorithm optimization flowchart	12
3.1	Diagram of the Flow of the System's Development	30
3.2	Overview of the Parser user Interface and logic	35
4.1	Key evaluator code snippet (constraint checking loop)	47
4.2	Enhanced ARO adaptation code:	49
4.3	Fitness Distribution by Algorithm (Box Plot)	54
4.4	Average Execution Time per Run	59
4.5	Convergence Analysis	60

LIST OF TABLES

TABLES	TITLE OF TABLES	PAGES
1.1	Objectives-Methodology Mapping Table	3
2.1	Benchmark Datasets and Evaluation Metrics Table	17
2.2	Algorithmic Approaches to UCTP Table	24
3.1	Controlled Experimental Environment Baseline	35
3.2	Defined Performance Metrics	36
3.3	Evaluation metrics	41
3.4	Cohen's d and Cliff's δ table	43
4.1	Software Requirements	45
4.2	Experimental Parameters	51
4.3	Overall Algorithm Performance	53
4.4	Best Fitness per Algorithm	55
4.5	Pairwise Mann-Whitney U Test Results	57
4.6	Cliff's Delta Effect Sizes for Pairwise Algorithm	58

ABBREVIATIONS

UCTP	University Course Timetabling Problem
NP	Non-deterministic Polynomial Time
GA	Genetic Algorithm
ARO	Artificial Rabbit Optimization
SA	Simulated Annealing
ACO	Ant Colony Optimization
ITC-2007	International Timetabling Competition 2007
AI	Artificial Intelligence
ML	Machine Learning
MIP	Mixed-Integer Programming
PSO	Particle Swarm Optimization
VNS-TS	Variable Neighborhood Search-Tabu Search
VNS	Variable Neighborhood Search
IWD	Intelligent Water Drops
HH	Hyper-Heuristic
CSV	Comma-Separated Values
ANOVA	Analysis of Variance
DSR	Design Science Research

ABSTRACT

University course timetabling is a Non-deterministic-polynomial Time(NP)-hard combinatorial problem of assigning courses, instructors, students, and rooms to timeslots under hard and soft constraints, for which exact methods are generally not feasible, and metaheuristics must be applied. A key unanswered question in the literature was how the continuous metaheuristic Artificial Rabbit Optimization (ARO) could be made practical for the discrete University Course Timetabling Problem (UCTP), given that the original ARO's vector-based operators were semantically incompatible with timetables. This study constructed an enhanced version of ARO, referred to as Enhanced ARO, by incorporating domain-specific discrete operators, such as Kempe chain perturbation, slot-swap local search, and greedy repair, as well as adaptive phase control based on the population feasibility rate. Enhanced ARO was implemented using a pluggable Python architecture and benchmarked against Standard ARO, Genetic Algorithm (GA), Simulated Annealing (SA), and Particle Swarm Optimization (PSO) on 24 ITC-2007 Track 2 instances within constrained evaluation budgets. Results showed that performances followed a statistically significant ordering of $SA > \text{Enhanced ARO} > GA > PSO > \text{Standard ARO}$, whereby Enhanced ARO achieved 59% fitness enhancement over Standard ARO and managed a feasibility breakthrough on one hidden instance that all other tested algorithms failed to pass, although all algorithms remained far from the ITC-2007 competition results. It was recommended that future work explore increased evaluation budgets, hybrid SA-Enhanced ARO combinations, and machine learning-based operator selection. In conclusion, discretizing the continuous metaheuristic successfully made ARO viable for UCTP, offering a generic approach to adapt continuous metaheuristics to combinatorial optimization domains and laying a basis for subsequent algorithmic extensions.

Keywords: University Course Timetabling, Artificial Rabbit Optimization, Metaheuristics, Combinatorial Optimization, Constraint Satisfaction, Comparative Analysis

CHAPTER ONE

INTRODUCTION

1.1. Background Information

University Course Timetabling (UCTP) is a critical and recurring challenge for higher education institutions involving assignment of courses, students, instructors, and rooms to time-slots while satisfying numerous hard(mandatory) and soft(quality) constraints. It is classified as a Non deterministic Polynomial time or NP-hard combinatorial optimization problem, which means there is no known exact algorithm or solution that can efficiently solve large instances within reasonable time frames (Abdipoor et al., 2023; Assi et al., 2018; Muklason et al., 2019).

To overcome this complexity, metaheuristic and hybrid metaheuristic algorithms for example, Genetic Algorithms (GA), Simulated Annealing (SA), Particle Swarm Optimization (PSO), and Ant Colony Optimization (ACO), are commonly used in multistage approaches that generate valid solutions and then refine them over time (Kenekayoro et al., 2016; Lewis & Thompson, 2015; Muklason et al., 2019)

The Artificial Rabbit Optimization (ARO) algorithm, inspired by rabbit foraging and hiding behaviors, is a promising new metaheuristic with strong performance in other optimization domains, although its application to University Course Timetabling Problem is still emerging (Wang et al., 2022; Anka et al., 2025). This project presents a reformulated Artificial Rabbit Optimization algorithm aimed to effectively solve the University Course Timetabling Problem (UCTP) it incorporates adaptive controls and focuses on a comparative benchmark of its performance to known metaheuristics for actual university course arrangement problem

1.2. Statement of the Problem

The desired output of an automated university course timetabling system is an optimum, feasible, and constraint-free schedule (Babaei et al., 2015). However, the UCTP is an NP-hard combinatorial optimization problem which can hardly be optimally solved by exact techniques due to its scalability problem, forcing the application of metaheuristics (Abdipoor et al., 2023). Although ARO has been shown as a high-performance optimizer for the continuous optimization problems in engineering fields (Wang et al., 2022), their inherent continuous search mechanism is semantically inconsistent with the discrete and highly constraint-ed feature of timetable. Common discrete mappings result in heavy search distortion that fails the algorithm for constraint feasibility (Abdipoor et al., 2023).

Thus, it is an open question whether Artificial rabbit Optimization Algorithm when converted to a discrete optimization algorithm, can compete with existing discrete metaheuristic algorithms such as Simulated annealing (Sylejmani et al., 2022) and Genetic algorithms. Thus far, there is no literature to study the behavior of ARO when tailored to the timetabling problems based on domain specific operators (such as Kempe chains and greedy repair mechanisms), in contrast with the typical ones, considering the issue of constrained computing resources. In the absence of the aforementioned comparative study, academic decision-makers would be unable to decide for ARO to handle high constraint conflicts and make efficient scheduling in practice.

1.3. Aim and Objectives of the Study

to Investigate the usability of enhanced Artificial Rabbit Optimization algorithm for solving University Course Timetabling Problem and to compare the performance of this adapted version with that of its continuous formulation and the well-known discrete metaheuristics.

The specific objectives of the study are:

- i. to examine the limitations of existing metaheuristics through structured Literature review
- ii. to refine and implement a reformulation of Artificial Rabbit Optimization algorithm tailored for University Course Timetabling Problem.

- iii. to evaluate the usability of the enhanced Artificial Rabbit Optimization Algorithm.
- iv. to benchmark the performance of the enhanced Artificial Rabbit Optimization algorithm against existing metaheuristics.

1.4. Methodology

A Design Science Research (DSR) methodology was adopted, proceeding through six phases: problem identification and motivation, definition of solution objectives, design and development of the Enhanced ARO with domain-specific discrete operators, demonstration through modular implementation, evaluation via controlled benchmarking on ITC-2007 Track 2 instances, and communication of findings. Table 1.1 shows the mapping of the objectives of the study to the above methodology.

Table 1.1: Objective-Methodology Mapping Table

S/N	OBJECTIVES	METHODOLOGY
i.	examine limitations of existing metaheuristic and hybrid algorithms for UCTP	Comprehensive literature review and systematic analysis of recent research (2015–2025)
ii.	Design and implement an enhanced ARO algorithm tailored for UCTP	Algorithm development: Adapt ARO with domain-specific strategies, adaptive parameter control, and hybrid local search and Code implementation
iii.	Evaluate the usability of the enhanced ARO	Feasibility rate analysis; per-instance performance tracking; Mann-Whitney U pairwise tests with Bonferroni correction + Cliff's delta effect size
iv.	Benchmark the performance of the enhanced ARO algorithm against existing metaheuristics	24 ITC-2007 Track 2 instances; 1,800 total runs (15 runs $\times$ 5 algorithms $\times$ 24 instances); adaptive budgets: Early 5,000, Late 10,000, Hidden 15,000 evaluations

1.5. Significance of the Study

The current project explores how an improved Artificial Rabbit Optimization (ARO)-based hybrid algorithm, which uses domain-specific timetable operators, can be formulated to address critical shortcomings in modern university course timetabling (UCTP) systems. This study proposes an advanced hybrid ARO framework that enhances the balance between exploration and exploitation, accelerates convergence, and improves satisfaction of soft constraints (Abdipoor et al., 2023; Bashab et al., 2023; Arias-Osorio & Mora-Esquivel, 2020).

The refined ARO algorithm incorporates domain-specific adaptations and hybrid local search methods to address weaknesses inherent in existing metaheuristics. This work aims to tackle that issue by designing a set of discrete operators that preserve the exploration-exploitation spirit of the original continuous ARO formulation, and offering a repeatable comparative study on the performance of ARO, and some widely-used discrete metaheuristics (GA, SA, and PSO) under controlled evaluation budgets. (Abdipoor et al., 2023; Bashab et al., 2023; Arias-Osorio & Mora-Esquivel, 2020).

The study does not aim to give a better University Course Timetabling solver; it investigates the question of whether or not it is possible to make ARO viable for the discrete scheduling problem, and under what circumstances it is possible to achieve success in feasibility breakthroughs that cannot be achieved by a continuous formulation.

The comparative benchmarking study provides valuable insights enabling decision-makers to assess algorithmic improvements and make informed choices about metaheuristic selection, ultimately fostering efficient resource utilization, stakeholder satisfaction, and operational effectiveness in higher education (Abdipoor et al., 2023; Bashab et al., 2023; Arias-Osorio & Mora-Esquivel, 2020).

1.6. Scope of the study

The overall project focuses on designing, analyzing and testing an improved variant of the enhanced Artificial Rabbit Optimization algorithm for the University Course Timetabling Problem. It is concerned with the optimization of course placement in timeslots and rooms, and with meeting a vast array of hard and soft constraints, including the prevention of schedule conflicts, exploiting available resources to the fullest, and supporting student, faculty, and administration preferences. To evaluate the effectiveness, efficiency, and flexibility of different methods, such as Genetic algorithm, Simulated Annealing, and other metaheuristics, the study examines and compares these methods based on predetermined datasets (standard datasets).

1.7. Project Outline

Chapter 1 introduces the problem, aim, and objectives; Chapter 2 reviews relevant literature and metaheuristic approaches. Chapter 3 details the research methodology, problem formulation, and experimental design; Chapter 4 covers implementation, benchmarking results, and discussion. Chapter 5 summarizes findings, and acknowledges limitations.

CHAPTER TWO

LITERATURE REVIEW

2.1. Preamble

In this chapter, a systematic literature survey on techniques, approaches and algorithmic methodologies to effectively solve the UCTP in the light of primarily meta-heuristics techniques which have developed through (2015-2026) are given a broad cover in order to lay a clear ground on how a developed version called an Improved Artificial Rabbit Optimization(ARO) could be formulated, designed, implemented and further to investigate its efficiency comparing with the existed most promising competitive algorithms.

2.2. Review of Relevant Concepts

All concepts related to this project are discussed in this section.

2.2.1. University Course Timetabling Problem (UCTP)

The problem of constructing university course timetables, known as the University Course Timetabling Problem (UCTP) is the task of allocating classes, faculty members, students, and rooms to time slots within an academic semester in accordance with the institution's policies with minimal resource waste (Abdipoor et al., 2023; Babaei et al., 2015; Chen et al., 2021). Since UCTP is a computationally NP-hard combinatorial optimization problem, exact algorithmic approaches may not be feasible for many instances since the computational cost for achieving an exact solution grows exponentially as the size of the problem instance increases (Abdipoor et al., 2023; Babaei et al., 2015). University timetabling is complicated by several challenging requirements. The courses must be scheduled for increasing number of students and within several curricular frameworks for the students in campus facility.

This must be achieved along with scheduling instructors considering the requirements, and availability of space in campuses facility (Chen et al., 2021; Alghamdi et al., 2020). The structure of universities and their specific timetabling rules may differ from one institution to another so each university must be able to build and maintain a scheduling policy tailored to its requirements. Otherwise, problems arising from the bad scheduling

will eventually cause damage, e.g., underutilized academic facilities, scheduling conflicts between the curriculum requirements and the actual classes, and time-consuming effort in rectifying the schedules by manually.

Figure 2.1 shows Flowchart of the heuristic algorithm steps for university course timetabling

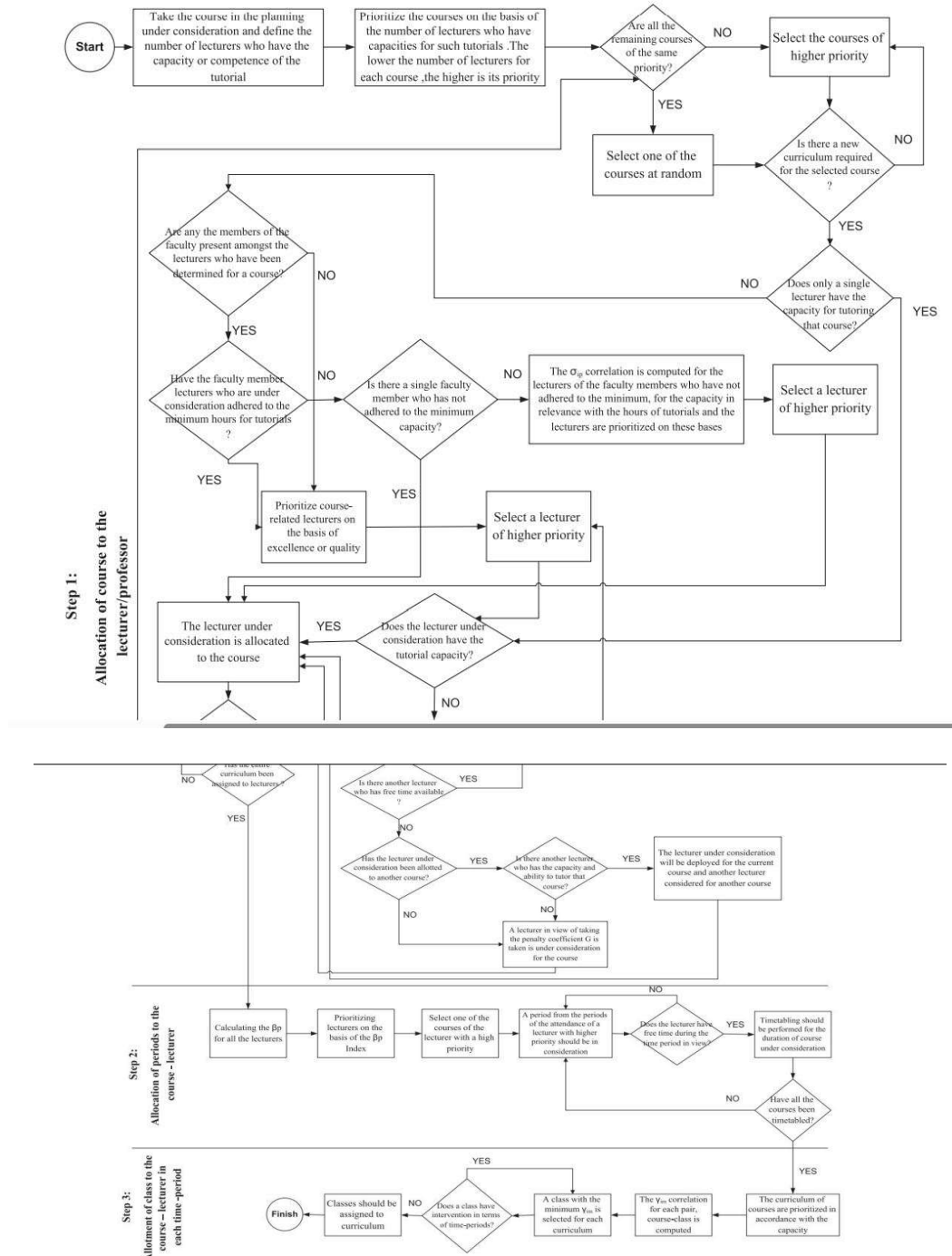


Figure 2.1: Flowchart of the heuristic algorithm steps for university course timetabling (adapted from Tavakoli et al., 2020).

2.2.2. Hard Constraints and Soft Constraints

The constraints of the problems are traditionally divided into two types, namely: hard constraints and soft constraints in the University Course Timetabling Problem (UCTP). Hard constraints are requirements that are indispensable in order to have a timetable that will be considered viable. Any solution should not be violated by any hard constraint, as it makes the solution not acceptable to be implemented (Abdipoor et al., 2023; Babaei et al., 2015; Sylejmani et al., 2022). Common hard constraints include ensuring that no student or instructor is assigned two events at the same time (the no-constraint), making sure that each venue is booked to only one event at a time, taking into account the availability of instructors and rooms, and ensuring that the seating capacity of booked rooms is adequate to accommodate the registered students (Babaei et al., 2015; Kaur & Saini, 2020). These limitations form the non-negotiable basis of any working schedule.

Soft constraints, on the other hand, are the preferences and quality requirements that are desirable and should be met at the maximum possible level, but have no impact on declaring a timetable as infeasible (Abdipoor et al., 2023; Babaei et al., 2015; Sylejmani et al., 2022). Rather, the violation of soft constraints are normally punished, and the optimization goal is to minimize the cumulative penalty. Soft constraints may include reducing idle time/gaps in schedules by students and instructors, supporting compact timetables, evenly distributing courses, avoiding overloading certain days, accommodating instructor preferences in specific timeslots or days, optimizing room use by not underutilizing or overmatching room capacity and size, and making related courses co-locate, such as lectures and tutorials (Babaei et al., 2015; Kaur & Saini, 2020; Chen et al., 2020).

The difficulty of the UCTP lies in the need to achieve feasibility: to satisfy all hard constraints, and, at the same time, optimize quality by minimizing the penalties, thus requiring advanced algorithmic approaches able to navigate in complicated trade-offs.

2.2.3. Metaheuristics

Metaheuristics are written algorithmic designs that guide the search in optimization problems, especially those which are computationally intractable to their exact solution (Abdipoor et al., 2023; Babaei et al., 2015; Alghamdi et al., 2020).

Metaheuristics also provide near-optimal solutions to large-scale problems that exact algorithms would demand prohibitive computational time, rather than optimum solutions, providing solutions through effective exploration of the solution space. Some of the most popular metaheuristic techniques used in the University Course Timetabling Problem (UCTP) include Genetic Algorithms (GA), which mimic the biological evolution process through the use of selection, crossover, and mutation; Simulated Annealing (SA), which is based on the metallurgical annealing process; Tabu Search (TS), which employs memory structures in order to avoid rediscovery of previously explored solutions; Particle Swarm Optimization (PSO), which builds on the collective behavior of bird flocks; and additional nature-inspired algorithms, such as those mentioned.

The University Course Timetabling Problem (UCTP) can be especially well-suited for metaheuristics due to the flexibility, adaptability, and ability to explore extensive and complex search spaces without requiring mathematical formulations specific to the problem. However, they also have their drawbacks, such as being sensitive to the setting of parameters, predisposed to early convergence, i.e., getting stuck in the local optima, and lack a guarantee that the globally optimal solution will be obtained (Abdipoor et al., 2023; Babaei et al., 2015; Alghamdi et al., 2020). To address these flaws, researchers have turned to hybrid approaches to combine several metaheuristics or combine metaheuristics with local search methods to improve exploration (exploration of the solution space) and exploitation (refinement and improvement of viable solutions) (Rezaeipanah et al., 2020; Vianna et al., 2020; Thepphakorn and Pongcharoen, 2020).

2.2.4. Hybrid Metaheuristics

Hybrid metaheuristics represent a sophisticated category of optimization algorithms, which combines the advantages of two or more metaheuristic or heuristic methods in an attempt to overcome the drawbacks of each of the original methods (Abdipoor et al., 2023; Kaur & Saini, 2020; Rezaeipannah et al., 2020). The rationale behind this is that the various algorithms are complementary, as some have global exploration qualities and others local exploitation. By combining these methods, hybrid algorithms aim to achieve high performance in terms of the quality of the solution, the rate of convergence, and the resistance to a range of problem examples (Vianna et al., 2020; Thepphakorn and Pongcharoen, 2020; Arias-Osorio and Mora-Esquivel, 2020).

Processual hybrid strategies most frequently reported in UCTP literature include the combination of Genetic Algorithms with local-search strategies like Hill Climbing or Tabu Search to produce new solutions; the combination of Simulated Annealing with Variable Neighbourhood Search (VNS) in order to balance between diversification and intensification; and the use of local-search methods in population-based algorithms, such as Cuckoo Search or Firefly Algorithm to refine solutions generated by GA (Rezaeipannah et al., 2020; Vianna et al., 2020; As an example, Rezaeipannah et al. (2020) have designed a hybrid algorithm, which was a combination of an improved parallel Genetic Algorithm and local search, producing significant gains in the quality of solutions and the speed of computations. Similarly, Vianna et al. (2020) proposed a hybrid VNS-TS (Variable Neighborhood Search-Tabu Search) algorithm that outperformed traditional methods as it offers a good balance between exploration and exploitation. Despite some benefits associated with hybrid metaheuristics, these approaches also have issues, such as the complexity of design, the need to carefully fine-tune various parameters, and a likely rise in computational expenses (Abdipoor et al., 2023; Kaur & Saini, 2020). Figure 2.2 shows the Genetic Algorithm optimization flowchart

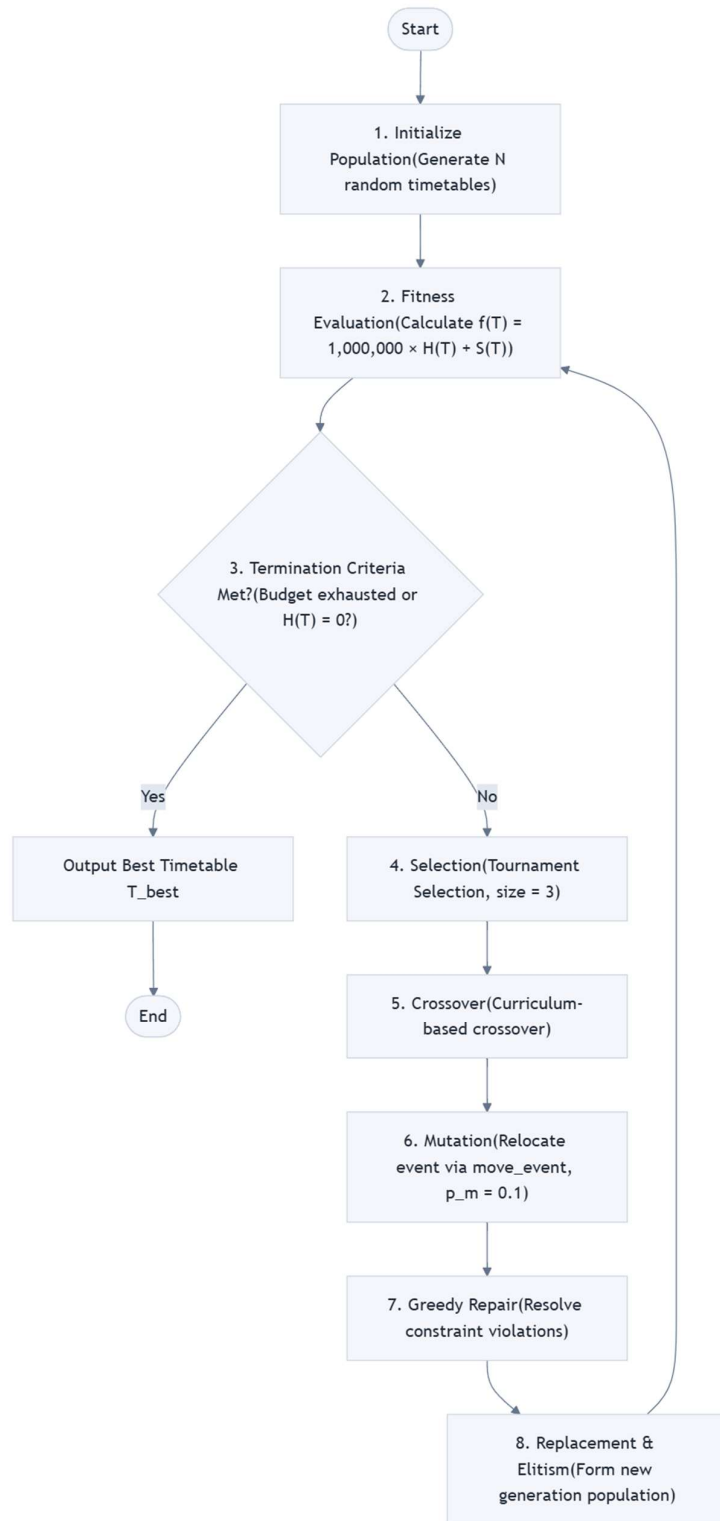


Figure 2.2: Genetic Algorithm optimization flowchart (adapted from Rezaeipannah et al. (2020))

2.2.5. Hyper-heuristics

Hyper-heuristics represent a further step of algorithm abstraction, focusing on the creation of mechanisms operating in a heuristics space instead of in the manipulated space (Abdipoor et al., 2023). The main idea is to automatically select, combine, or generate an appropriate low-level heuristic for a particular problem scenario, thereby reducing the need for human experience to design an algorithm and to set up parameter values. The two categories of hyper-heuristics mostly consist of selection: heuristics that are part of an existing repertoire of selection hyper-heuristics that choose between existing heuristics based on their performance; and generation: heuristics that create new heuristics by combining parts of existing ones (Abdipoor et al., 2023; Kaur & Saini, 2020).

Hyper-heuristics have been attractive to the UCTP due to their ability to be generalized and adaptable. Unlike traditional meta-heuristics, which can be highly problem-specific, hyper-heuristics aim to achieve robustness when faced with diverse cases of problems, and do so without having to reconfigure the hyper-heuristic to the same level (Abdipoor et al., 2023). As an example, a selection hyper-heuristic may make use of a reinforcement learning process or some kind of adaptive process to decide which low-level heuristic, such as swapping courses, relocating events, or performing Kempe chain operations, is most effective at specific points in the search. There is empirical evidence that hyper-heuristics are able to reach performance that is competitive on benchmark UCTP datasets, which might support their practical implementations (Abdipoor et al., 2023; Kaur & Saini, 2020). However, hyper-heuristics also have a set of problems, such as the computational cost of learning or fine-tuning heuristic choice strategies and the need to have highly curated sets of heuristics that provide sufficient diversity and complementation.

2.2.6. Artificial Rabbit Optimization (ARO)

ARO is a swarm intelligence metaheuristic introduced by Wang et al. (2022) inspired by predator-escape behavior in rabbits. The algorithm maintains a population of candidate solutions (rabbits) and updates them through two core mechanisms: detour foraging (exploration) and random hiding (exploitation).

Standard ARO Mathematical Model:

The original ARO operates on continuous position vectors in R^d . The two core update equations are:

Detour Foraging (Exploration):

$$x_i(t+1) = x_{\text{best}}(t) + L * (x_{r1}(t) - x_{r2}(t)) \quad (1)$$

Random Hiding (Exploitation):

$$x_i(t+1) = x_{\text{best}}(t) + C(t) * (x_{r1}(t) - x_{r2}(t)) \quad (2)$$

Where:

- x_{best} is the best solution found so far
- x_{r1} and x_{r2} are two randomly selected distinct solutions
- L is the learning coefficient (typically 2.0)
- $C(t)$ is the escaping coefficient that decreases over time from 1.0 to 0.1

Discretization Mapping for UCTP:

Since room and timeslot allocations are discrete integers, standard ARO cannot be applied directly. The naive discretization maps continuous positions to discrete assignments:

For each event e_i :

$$\begin{aligned} r_i &= \text{floor}(x_i * |R|) \bmod |R| \\ p_i &= \text{floor}(x_{\{i + |C|\}} * |P|) \bmod |P| \end{aligned} \quad (3)$$

Where x_i in $[0,1]$ is the continuous position component. This mapping is semantically fragile: fractional updates can produce invalid indices, and Euclidean distance between continuous vectors does not correspond to meaningful timetable distance.

Limitations of Standard ARO on UCTP:

- i. Vector perturbations ignore timetable structure
- ii. Constraint repairs are crude (random reassignment) and ineffective
- iii. No domain knowledge is embedded in algorithm moves
- iv. Feasibility rate remains low even with many evaluations
- v. Continuous operators cannot preserve discrete feasibility

2.2.7. Benchmark Datasets and Metrics of Evaluation.

Datasets represent a notable part of UCTP studies since they provide standardized problem instances, which make it possible to objectively compare the algorithmic strategies (Abdipoor et al., 2023; Babaei et al., 2015; Kaur & Saini, 2020). The most commonly mentioned datasets in the UCTP literature are the ITC-2007 collection (International Timetabling Competition 2007), including real-life instances of several universities with different types of constraints; the Socha dataset, with its variety of sizes and complexities; and the BenPaechter dataset, with its variety of small and large instances that are commonly used in early timetabling studies (Babaei et al., 2015; Kaur & Saini, 2020; Bettinelli et al., 2015).

Metrics used to evaluate UCTP algorithms are usually solution quality, which is determined by the sum of the score of soft-constraint violation (a lower score indicates higher quality); feasibility, meaning whether all hard constraints have been met; algorithm computational time, a measure of algorithm efficiency; and robustness, a measure of consistency in performance by the algorithm on many runs and problem instances (Abdipoor et al., 2023; Kaur & Saini, 2020; Davison et al., 2024). Other studies consider multi-objective criterion, including the trade-off between student satisfaction and resource usage or reducing the penalty cost and disruption of the schedule (Davison et al., 2024). Reproducibility, transparency, and equitable comparison of research contributions is achieved as standardized benchmarks and metrics are used, thus enhancing the field and identification of best practices. Table 2.1 shows the Benchmark Datasets and Evaluation Metrics Table

Table 2.1: Benchmark Datasets and Evaluation Metrics Table

Dataset/Competition	Description	Usage in Literature	Citation
ITC-2007	Large, complex, real-world university timetabling benchmark	Standard for algorithm testing/comparison	Kaur & Saini, 2020; Abdipoor et al., 2023
Socha Dataset	Diverse problem sizes, early benchmark	Used for algorithm development	Babaei et al., 2015
BenPaechter Dataset	Small to medium instances, historical	Foundation for early solutions	Bettinelli et al., 2015
ITC-2019	Latest competition, modern constraints	Evaluates method robustness, efficiency	Sylejmani et al., 2022
Penalty Score	Quantifies soft constraint violations	Standard quality metric	Kaur & Saini, 2020

	Measures runtime efficiency	Algorithm speed evaluation	Davison et al., 2024
--	-----------------------------	----------------------------	----------------------

Computational Time			
Robustness	Consistency across multiple runs/instances	Checks stability of algorithms	Davison et al., 2024

2.2.8. Artificial Intelligence and Machine Learning in timetabling.

The application of machine-learning (ML) and artificial-intelligence (AI) tools to timetabling research is a new and potentially promising direction towards enhancing the work of the algorithms and automating the task (Ghaffar et al., 2025). ML strategies have the ability to learn from past timetabling information, to forecast the effectiveness of different heuristics or parameter configurations, and can dynamically guide the search based on the characteristics of the problem.

Modern studies have also explored deep-learning solutions to extract features and predict outcomes in timetabling, and have also investigated hybrid models that combine traditional meta-heuristics with machine-learning features to perform an adaptive adjustment of the parameters or addressing constraints (Ghaffar et al., 2025). Furthermore, the AI methodologies, constraint programming, and knowledge representation, have been combined with optimization techniques to address the complex timetabling problems in a better way. Although recent literature on the implementation of ML and AI to UCTP is quite early when compared to conventional meta-heuristics, the benefits can be seen in the less reliance on manual parameterization, the ability to generalize to new problem cases, as well as sustained learning under new data. However, there are still problems with data availability, the computational cost of model training, and understanding of strategies learned (Ghaffar et al., 2025).

2.3. Review of Related Methods

The techniques used in the literature are discussed in this part.

2.3.1. Genetic Algorithm-Based Approaches

University Course Timetabling Problem (UCTP) has broadly been tackled using genetic algorithms (GAs) because of the inherent versatility and ability to deal with complex constraint structures (Babaei et al., 2015; Rezaeipannah et al., 2020; Arias-Osorio and Mora-Esquivel, 2020).

Rezaeipannah et al. (2020) proposed the best parallel genetic algorithm (IPGALS), which has a new distance to feasibility measure. This indicator directs the optimization of the search to possible solutions and at the same time maximizes soft constraints. This parallel structure allows different populations to evolve independently, and an exchange of information periodically increases genetic diversity and helps to prevent early convergence. ITC benchmark There has been experimental analysis on ITC-2007 benchmark cases indicating that IPGALS is competitive in quality of solutions and computational efficiency compared to state-of-the-art methods.

Mora-Esquivel and Arias-Osorio (2020) presented a variant of a hybrid GA that includes local-search visitors like hill climbing and tabu search to narrow down GA-generated solutions. Their approach makes it clear that they need to achieve a balance of the exploration process (third category: crossover and mutation) and the exploitation process (fourth category: local search) to improve convergence behavior and quality of solutions. The investigation underlines that pure GAs frequently fails to fine-tune solutions, and with hybridization, they may be highly beneficial.

However, GA-based algorithms have a few limitations, such as being sensitive to parameter selection (population size, crossover rate, mutation rate), the computational cost associated with repeated fitness calculations, and that they can easily converge to a local minimum in case of the under-preservation of diversity (Babaei et al., 2015; Rezaeipannah et al., 2020).

2.3.2. Simulated Annealing and Tabu Search

Annealing and Tabu search are methods used to simulate the actual process of annealing and searching respectively.

The local search meta-heuristics that have already been applied to UCTP are simulated annealing (SA) and tabu search (TS) (Sylejmani et al., 2022; Vianna et al., 2020; Babaei et al., 2015).

Sylejmani et al. (2022) suggested a two-phase SA methodology that would involve penalization policies to the ITC-2019 competition. The first stage focuses more on feasibility with extreme high penalties on hard constraint violations, with the second stage taking more lax constraints as the acceptability criterion. In the competition, this method achieved competitive positions, and hence proves the effectiveness of adaptive penalty mechanisms and temperature schedules in controlling the search process.

Single-solution metaheuristic SA uses a geometric cooling schedule: it begins with a greedy initialized solution, applies the swap_events to a solution in every iteration, and accepts worse solutions with a probability $\exp(f / T)$, where $T(k) = T_0 \times \alpha^k$, T_0 , where T is a constant 100, and $\alpha = 0.995$. SA as a single-solution algorithm performs far more iterations than the population-based algorithms with same budget: this explains SA's average performance, more iterations mean more detailed local search and greater possibility to escape from local optimum using the Metropolis criterion (Kirkpatrick et al., 1983).

Vianna et al. (2020) proposed a UCTP hybrid Variable Neighborhood Search (VNS) and TS (VNS-TS). VNS component systematically examines different neighborhood structures to avoid local optima and TS uses memory structures to avoid cycling and recombining the already known solutions. The hybrid formulation compares well with both manual timetabling and single-method algorithms in the quality of solutions and is also flexible to different instances of problems. The analysis highlights that both diversification and intensification (VNS and TS) cannot work as a team without integration that can yield strong performance. Nevertheless, the SA and TS methods assume careful parameter adjustment, e.g., cooling schedules, initial temperature, tabu

tenure, and neighborhood definitions, which are time-consuming and much more problem-specific (Babaei et al., 2015; Sylejmani et al., 2022; Vianna et al., 2020).

2.3.3. Swarm Intelligence and Nature-Based Algorithms.

Swarm intelligence algorithms based on collective behaviors in nature have attracted significant attention to research on UCTP because of their conceptual simplicity and empirical usefulness (Thepphakorn and Pongcharoen, 2020; Aldeeb et al., 2021).

When applied to UCTP, Thepphakorn and Pongcharoen (2020) proposed the use of performance improvement strategies to the Cuckoo Search (CS) algorithm. Their hybrid CS includes local-search techniques, adaptive parameter laws, and diversity-preservation measures, thus solving the conventional CS problems of sluggish convergence and inadequate exploitation. The evaluation of the experiments

conducted on benchmark data has shown that hybrid CS is more effective than standard CS, Particle Swarm Optimization (PSO), and simplistic GAs in terms of solution quality and convergence speed.

The hybrid Intelligent Water Drops (IWD) algorithm was developed by Aldeeb et al. (2021) to study the examination timetabling - a related issue to UCTP. The IWD model is used in simulating water drop movement through riverbeds by using soil and velocity concepts to direct the search. The hybrid version combines local-search algorithms and adaptive strategies to aim at increased exploitation. Benchmark testing shows that the IWD version attains competitive performance, highlighting the possibilities of nature-inspired algorithms to the problem of timetabling.

Although the approach of swarm intelligence has the benefits of simplicity and parallelizability, the approach also faces difficulties involving the parameter tuning, speed of convergence, and exploration versus exploitation (Thepphakorn & Pongcharoen, 2020; Aldeeb et al., 2021).

2.3.4. Hyper-heuristic Approaches

Hyper-heuristic methods have become an effective approach towards UCTP, with the aptitude of the full automation of the dynamic (now viewpoint of heuristics) inclusion or creation of low-level heuristics in the procedure of locating a solution (Abdipoor et al., 2023; Kaur & Saini, 2020).

Abdipoor et al. (2023) introduced a constructive hyper-heuristic which chooses dynamically based on the performance record construction heuristics in a pool according to their performance and the state of the problem at hand. The hyper-heuristic uses a scoring system to evaluate the effectiveness of each heuristic and varies its selection procedure on-line. Assessment on ITC-2007 datasets indicates that the hyper-heuristic attains results that are similar to problem-specific meta-heuristics, in addition to being more generalized despite varying problem instances.

The major benefit of hyper-heuristics is that they are domain-independent and they do not require many problem-specific adjustments. Hyper-heuristics are much more applicable to various timetabling situations operating at a greater degree of abstraction with only slight adjustments. However, the effectiveness of hyper-heuristics is highly related to the quality and variety of the low-level heuristic pool, the selection or learning mechanism that informs the use of heuristics (Abdipoor et al., 2023; Kaur & Saini, 2020). Future studies involve extending to the use of machine-learning methods to narrow heuristic selection and studying generative hyper-heuristic methods that can create novel heuristics adapted to the specifics of the problem.

2.3.5. Mathematical Programming and Matheuristics

Mixed-integer programming (MIP) and matheuristics (combinations of mathematical programming with heuristics) represent exact and hybrid approaches to UCTP (Rappos et al., 2022; Davison et al., 2024; Bettinelli et al., 2015). Rappos et al. (2022) developed a mixed-integer programming model for UCTP that precisely captures hard and soft constraints, using commercial solvers such as CPLEX to find optimal solutions for small to medium-sized instances. For larger instances where exact solutions are computationally prohibitive, they proposed a two-stage approach combining MIP for initial feasibility with local search for improvement. The combination of hybrid algorithms performed very high when assessed on internationally renowned timetabling challenges proving the strength of integrating heuristics and exact methods. The recent issue of hybrid teaching, combining on-campus and distance learning environments for university education, in university timetabling was tackled by Davison et al. (2024), who formulated a mixed integer linear programming model with new modelling considerations concerning the capacity of rooms, needs in technology and preferences of students and coupled it with adaptive local search as their hybrid approach. They developed robust algorithms that yield valid schedules even for complex real-life applications.

The academic domain specific curriculum-based course timetabling has been surveyed in Bettinelli et al. (2015) together with the range of applied mathematical modelling approaches, including column generation, branch-and-price and integer programming.

MIP-based optimization could yield exact solutions for tiny instances and deliver good solutions in larger instances when integrated into metaheuristics. In spite of this fact, this technique may lead to poor scaling performance (polynomial time solutions grow in exponential fashion with respect to problem dimensions) when used as pure optimization, for any problem apart from the extremely small size (Rappos et al., 2022; Davison et al., 2024; Bettinelli et al., 2015). Table 2.2 Shows all algorithmic approaches to UCTP

Table 2.2: Algorithmic Approaches to UCTP Table

Approach/Method	Key Features	Strengths	Limitations	Example References
Genetic Algorithms (GA)	Evolutionary operators (selection, crossover, mutation)	Flexible, handles large spaces	Parameter sensitivity, risk of local optima	Babaei et al., 2015; Rezaeipanah et al., 2020
Simulated Annealing (SA)	Probabilistic acceptance criteria, cooling schedule	Simple, effective for local search	Slow convergence, requires tuning	Sylejmani et al., 2022
Tabu Search (TS)	Memory structures prevent cycling	Good at escaping local optima	Sensitive to tabu tenure	Vianna et al., 2020
Hybrid Metaheuristics	Combines two/more algorithms (e.g., GA+LS, VNS-TS)	High solution quality, robust	Complex design and parameter tuning	Rezaeipanah et al., 2020; Vianna et al., 2020
Hyper-heuristics	Select/generate heuristics adaptively	Adaptive, generalizable	Requires diverse heuristic pool	Abdipoor et al., 2023

Swarm Intelligence	Population-based (e.g., Cuckoo, Bees, IWD)	Simple, parallelizable	Parameter sensitivity, convergence	Thepphakorn & Pongcharoen, 2020; Aldeeb et al., 2021
Mathematical Programming (MIP)	Exact constraint modeling, solver-based	Precise, guarantees optimality	Scalability, computational expense	Rappos et al., 2022; Davison et al., 2024
Machine Learning/AI	Data-driven/adaptive (ML, RL, deep learning)	Learns patterns, adaptive	Data needs, computational cost	Ghaffar et al., 2025
Artificial Rabbit Optimization	Nature-inspired population search, hybrid-ready	Fast convergence, robust	Still emerging, requires adaptation	Ghaffar et al., 2025

2.3.6. Emerging AI and Machine Learning Approaches

The utilization of advanced AI and machine learning models in UCTP is an emerging area of literature that highlights the new era of data-driven and intelligent approaches for this optimization problem (Ghaffar et al., 2025). Ghaffar et al. (2025) conducted a systematic review on hybridization and AI techniques in examination timetabling, they indicated several trends including deep learning for recognizing patterns and relationships, reinforcement learning for adaptively selecting heuristic agents, and artificial neural networks for predicting constraint feasibility. The authors also concluded that AI-aided techniques are generally promising for automatic exam timetabling, adaptability and improved performance, nevertheless the majority of these solutions are still at nascent stages and need further empirical evaluation for various problem instances.

2.4. Summary of Literature Review

As evidenced in the literature review within this chapter, the University Course Timetabling problem continues to be an active and compelling area of research and saw great advances between 2015-2025. Due to the NP-hard nature of the UCTP, exact solution methodologies become computationally prohibitive for realistic size instances and metaheuristic, hybrid and hyper-heuristic algorithms have become more prominent methods. While still the foundation of many approaches to problems in the class, more traditional metaheuristic methods like the Genetic Algorithm, Tabu Search and Simulated Annealing generally suffer from drawbacks such as: parameter tuning, early convergence, and balance of exploitation and exploration. Hybridization of these techniques to capture advantages from more than one optimization technique have proven very successful, with papers such as Rezaeipanah et al. (2020), Vianna et al. (2020) and Thepphakorn and Pongcharoen (2020) showcasing that the combination of population-based metaheuristic algorithms with local improvement routines such as greedy search and local optimizations significantly enhances the quality and robustness of the obtained solutions. Hyper-heuristics represent an approach to automated algorithm design for solving problems by exploring the search space at a higher level of abstraction Abdipoor et al. (2023). Mathematical Programming based approaches such as mathematical models solved with linear, integer and quadratic optimizers or via methods known as ‘matheuristics’ can achieve optimal solutions on smaller instances of UCTP and provide strong results for large problems with additional heuristics included.

(Rappos et al., 2022) There is also recent literature indicating a rise in interest in the usage of the novel meta-heuristic Artificial Rabbit Optimization and more broadly Artificial intelligence and machine learning based optimizations to the problem. (Ghaffar et al., 2025) While this research, will investigate the application of Artificial Rabbit Optimization (ARO) on the UCTP, the existing literature does not discuss sufficient research regarding the optimization of ARO parameters specific for the UCTP or integrate its application with the highly cited optimization schemes such as hybrid and hyper-heuristics through development of new heuristics tailored for the UCTP; these areas of the current literature this research addresses via implementing a new adaptive ARO based approach to optimize parameters.

CHAPTER THREE

RESEARCH METHODOLOGY

3.1. Preamble

In this Chapter, the design and algorithm analysis of the UCTP solver programmed by this paper will be presented. This program can read ITC-2007 Track 2 timetabling problems, quantify the hard and soft constraint conditions, make comparison between Enhanced ARO algorithm and Standard ARO algorithm, GA, SA algorithm, and PSO algorithm. It includes the system flow, requirements specification, architecture specification, physical design specification, logical design specification, constraint model specification, Enhanced ARO algorithm process improvement specification, and Benchmark specification.

3.2. System Overview

This research adopts a Design Science Research approach that spans from problem definition to the design of a solution, controlled experiment and documentation of the research. An extensible framework was design with data ingestion, problem representation, fitness functions, neighborhood operators and search algorithm solvers working together in a timetable optimizations pipeline. With the layered architecture described above, it is possible to make a distinct representation of a continuous metaheuristic and also to assure fairness to comparisons by sharing identical test datasets and search budget, and employing common statistical approaches for evaluation, and ensuring reproducibility through static random number generators and consistent definitions for fitness. Focus is given to both feasibility increase and rate of convergence instead of achieving optimality.

3.3. Architectural Design

The overall structure of the workflow is sequential experimental execution. Data files are parsed to a internal problem formulation; candidate schedules are generated by each of the metaheuristic solvers; hard violation and soft penalty is evaluated by the shared evaluator, and the comparison data is logged by the benchmark engine to obtain and visualize statistics.

The architecture is divided into five layers. The Data Layer reads benchmark instances. The Problem Representation Layer stores rooms, courses, students, curricula, and flattened lecture events. The Optimization Core contains the evaluator and neighborhood operators. The Algorithm Layer contains Standard ARO, Enhanced ARO, GA, SA, and PSO. The Benchmark and Analysis Layer runs experiments, saves CSV files, and creates graphs.

- a. Data Layer: The Data Layer is responsible for parsing, validating, and storing problem instances in memory
parser.py converts dataset files into ProblemInstance objects.
- b. Representation Layer: problem.py defines rooms, courses, students, curricula, and event mappings.
- c. Evaluation Layer: evaluator.py computes hard violations, soft penalties, total fitness, and actual evaluation counts.
- d. Operator Layer: operators.py implements move, swap, Kempe-chain perturbation, slot-swap local search, and greedy repair.
- e. Solver Layer: solvers.py implements the five optimization algorithms.
- f. Benchmark Layer: benchmark.py, main.py, analyze_results.py, and analyze_extended.py run experiments and produce results Figure 3.1 shows a diagram of the Flow of the System's Development.

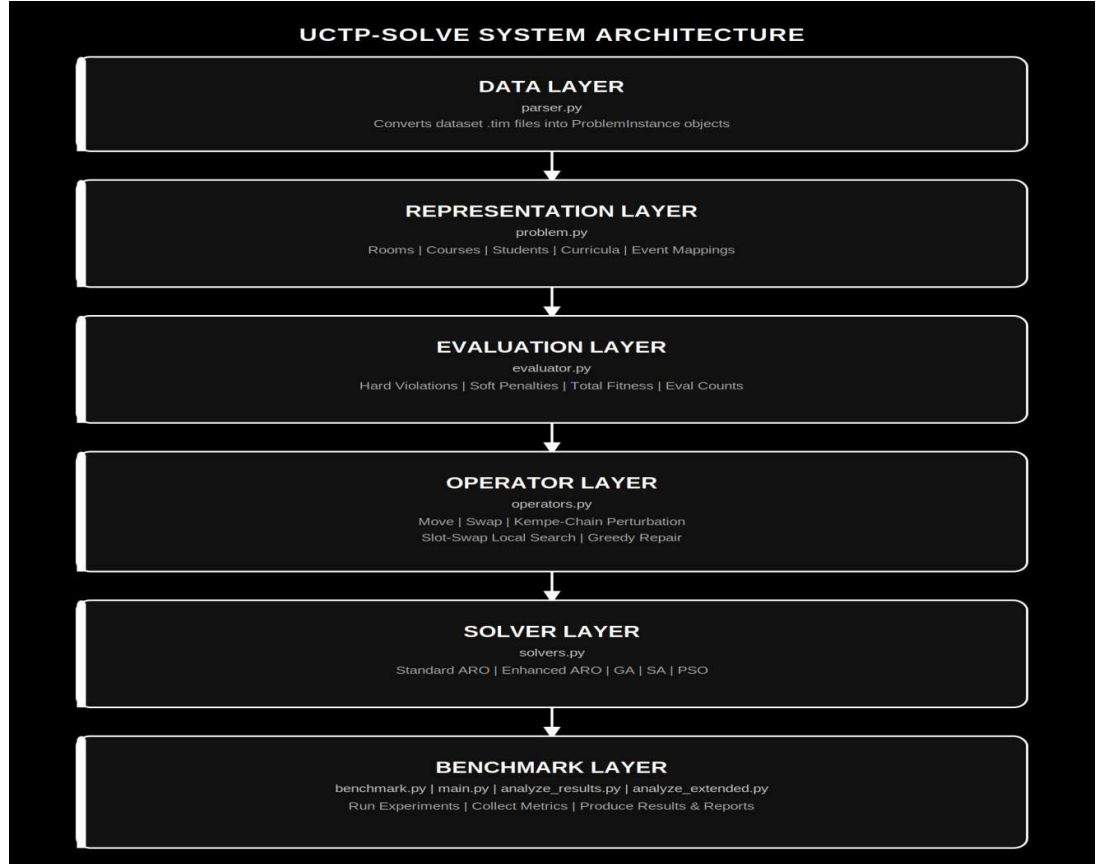


Figure 3.1: A diagram of the Flow of the System's Development

3.3.1. The University Course Timetabling Problem

Given a set of constraints and resources, UCTP requires assigning each course event to a room-timeslot pair. Formally, we define:

C: Set of all course events (lectures), $|C| = n$ (typically 200-400 events)

R: Set of available rooms, $|R| = m$ (typically 10-20 rooms)

P: Set of available timeslots, $|P| = t$ (typically 45 slots: 5 days $\times$ 9 periods)

S: Set of students enrolled in courses

T: Curriculum groups (groups of students taking similar courses)

Ω_{hard} : Set of hard constraints (mandatory, must be satisfied)

Ω_{soft} : Set of soft constraints (preferences, minimize violations)

A valid timetable T assigns each course event $c \in C$ to exactly one room $r \in R$ and one timeslot $p \in P$, denoted as $T(c) = (r, p)$.

3.3.2. Hard Constraints

Hard constraints define feasibility. A timetable is feasible if and only if it violates zero hard constraints. The seven hard constraints in ITC-2007 Track 2 are:

- i. No Room Conflicts: Each room can host at most one event in any given time slot.

For all pairs (c_i, c_j) where $T(c_i). \text{room} = T(c_j). \text{room}$: $T(c_i). \text{time slot} \neq T(c_j). \text{time slot}$

- ii. No Teacher Conflicts: Each lecturer teaches at most one course per time slot.

For all courses taught by the same instructor: no two can occur in the same time slot

- iii. No Student Curriculum Conflicts: Students enrolled in both courses cannot attend both if scheduled at the same time

For all pairs of courses in the same curriculum: different time slots.

- iv. No Same-Course Period Conflict: different lecture events belonging to the same course must not be placed in the same timeslot.

For any two-lecture events e_i and e_j of the same course: $\text{period}(e_i) \neq \text{period}(e_j)$.
Each lecture occurrence is still assigned exactly one room-timeslot pair by the timetable encoding

- v. Room Capacity Sufficient: Room capacity must accommodate all students in the course.

For each course c : $\text{room_capacity}(T(c). \text{room}) \geq \text{student_count}(c)$

- vi. Allowed Timeslots: Courses can only be scheduled in their allowed timeslots

For each course c : $T(c). \text{timeslot} \in \text{allowed_timeslots}(c)$

Room Restrictions: if a course has a restricted set of allowable rooms, the assigned room must belong to that set.

For each event e of course c : if $\text{allowed_rooms}(c)$ is non-empty, $\text{room}(T(e))$ must be in $\text{allowed_rooms}(c)$.

3.3.3. Soft Constraints (Minimize)

Soft constraints define solution quality. The four soft constraints in ITC-2007 Track 2 are:

- i. **Room Utilization:** Minimize unused room capacity (preference for smaller appropriate rooms).

Penalty for each event:

$$\text{floor}(\max(0, \text{room_capacity} - \text{student_count}) / 10).$$

This rewards rooms that are large enough without wasting excessive capacity.

- ii. **Minimum Working Days:** lectures of the same course should be spread across enough teaching days.

Penalty:

$$\text{if } \text{daysused}(c) < \min(\text{lectures}(c), 3) \text{ add } 5 * (\min(\text{lectures}(c), 3) - \text{daysused}(c))$$

- iii. **Curriculum Compactness:** curriculum events should occur close together, with penalties for gaps between consecutive curriculum periods.

- iv. **Room Stability:** lectures of the same course should use as few different rooms as possible.

Penalty: $3 \times (\text{number_of_distinct_rooms_used_by_course} - 1)$.

3.3.4. Solution Encoding

Each solution is represented as a complete assignment matrix where each course maps to exactly one room-timeslot pair (Babaei et al., 2015; Kaur & Saini, 2020). This differs from vector-space representations by maintaining structural integrity of timetable constraints throughout optimization, avoiding the semantic fragility of continuous-to-discrete mappings noted by Wang et al. (2022). Timetable Encoding:

$$T \in \{(r,p) : r \in R, p \in P\}^{|C|} \quad (4)$$

Each course $c \in C$ has exactly one entry: $T[c] = (\text{room_id}, \text{timeslot_id})$.

Search space size:

$$(|R| \times |P|)^{|C|} = (20 \times 45)^{400} = 900^{400} \quad (5)$$

For a typical instance, this discrete encoding differs from vector-space representations. Instead of mapping to continuous $[0,1]^n$ space, we maintain the structural integrity of timetable constraints throughout optimization.

Feasibility and Penalty Calculation

Fitness evaluation combines hard and soft penalties:

$$f(T) = 1,000,000 \times H(T) + S(T)$$

Where:

$H(T)$ = number of hard constraint violations

$S(T)$ = weighted sum of soft constraint violations

The coefficient 1,000,000 ensures hard constraints dominate

The large weighting factor ensures that any feasible solution ($H=0$) has lower fitness than any infeasible solution ($H>0$).

3.3.5. Key Assumptions

- i. **Assumption 1:** Complete Information Availability. All course data is available before optimization begins. Student enrollments are known and fixed. Room inventories and features are predetermined. Instructor availability is known (Babaei et al., 2015).
- ii. **Assumption 2:** Discrete Optimization Space. Course-room-timeslot assignments are discrete. No partial assignments or fractional allocations. Search space contains all feasible and infeasible solutions (Babaei et al., 2015; Abdipoor et al., 2023).
- iii. **Assumption 3:** Fixed Constraint Weights. Hard constraint violations are weighted uniformly (1,000,000 penalty each). Soft constraint penalties are pre-specified and static. Weights do not adapt during optimization (Babaei et al., 2015; Kaur & Saini, 2020).
- iv. **Assumption 4:** The problem instance remains stationary throughout the entire search; courses cannot be added dynamically, the rooms will not be added, removed or re-configured; the time slots assigned are static throughout the search (Babaei et al., 2015; Abdipoor et al., 2023).
- v. **Assumption 5:** Evaluation Budget Constraint. Optimization operates under a fixed counted evaluation budget. All algorithms are compared using actual timetable evaluations, including evaluations spent inside repair and local-search routines. The benchmark uses adaptive caps according to instance size, typically 5,000 evaluations for 400-event instances and higher caps for smaller instances where repair is less expensive. Davison et al. (2024) and Kaur & Saini (2020).

3.4. Dataset Design

This talks about all data used, the parser as well as method of benchmarking

3.4.1. Dataset Format and Parser logic

The ITC-2007 Track 2 instances are parsed from standard competition files following the binary token-stream format established by Di Gaspero, McCollum, and Schaerf (2007). The parser reads seven sequential data blocks: room capacities, student-event enrollment matrix, event-feature requirements, room-feature provision matrix, event-period availability, and event-event curriculum relationships, with a fixed 45-period structure (5 days $\times$ 9 timeslots) across all instances. From these blocks, the parser constructs room objects with capacity attributes, course objects with teacher and student counts, student

enrollment lists, and curriculum groups by detecting shared student enrollments between events. Period constraints and room constraints are derived from availability and feature matching matrices respectively. This parsing approach follows the standardized benchmark ingestion practices recommended by (Abdipoor, Yaakob, Goh, & Abdullah 2020) and (Davison et al., 2024), who emphasize reproducible dataset handling for fair algorithmic comparison. The parser also supports JSON and CSV alternative formats for institutional data flexibility, enabling extension beyond the competition benchmark as advocated by recent survey literature on educational timetabling benchmarks (Ceschia, Di Gaspero, & Schaerf, 2023). Figure 3.2 shows the parser user interface

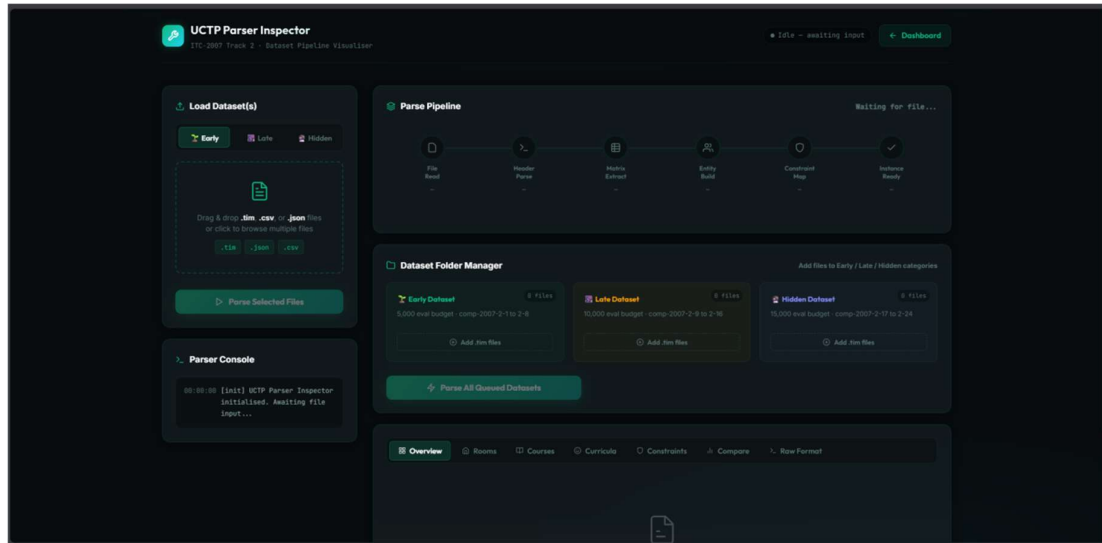


Figure 3.2: The Parser Interface & Logic (adapted from (Ceschia et al., 2023))

3.4.2. Experimental Environment

All experiments were conducted on a single controlled machine to eliminate hardware variance as a confounding factor.

Table 3.1 represents all controlled experimental environment baseline

Table 3.1: Controlled Experimental Environment Baseline

Parameter	Specification	Reproducibility Impact
Processor Architecture	AMD Ryzen 9 5000 series	Fixed architecture
Base Clock	3.3 GHz	Variable (boost clock not disabled)
Core Count	8	N/A
RAM Capacity	16 GB DDR4	Sufficient for all algorithms
RAM Speed	Not specified	Minor impact
Storage	1TB SSD	Fast I/O
Operating System	Windows 11, 64-bit	Fixed OS
Execution Language	Python 3.10+ (CPython)	Interpreted, single-threaded
Compiler Flags	N/A (Python is interpreted, not compiled)	—
Parallel Execution	Disabled (Python GIL)	All algorithms equally affected
Background Processes	Not controlled	Potential minor variance

3.4.3. Evaluation Performance Metrics

Three primary metrics were employed to ensure fair algorithmic comparison: (1) solution quality, computed as $f(T) = 1,000,000 \times H(T) + S(T)$, where $H(T)$ denotes hard constraint violations and $S(T)$ denotes soft penalty, aggregated into a single composite score via the shared evaluator; (2) feasibility rate, defined as the proportion of runs achieving $H(T) = 0$ and measured through post-hoc analysis; and (3) operational cost, quantified by counting total fitness evaluations (`evaluator.evaluation_count`) to enforce equal computational budgets across all algorithms. Secondary metrics included execution time measured via Python's `time.time()` function in seconds per run. Acknowledged limitations of the evaluation framework include the lack of convergence trajectory logging

over evaluation time and the absence of memory footprint measurements, which constrained deeper analysis of resource utilization patterns. Table 3.2 represents the defined Performance Metrics

Table 3.2: Defined Performance Metrics

Metric	Definition	Measurement	Unit	Priority
Solution Quality	$f(T) = \frac{1,000,000}{H(T) + S(T)}$	Single-pass evaluator	Composite score	Primary
Feasibility Rate	Proportion of runs with $H(T) = 0$	Post-hoc analysis	Percentage	Primary
Operational Cost	Counted fitness evaluations	evaluator.evaluation_count	Count	Primary (fairness)
Execution Time	Wall-clock time per run	Python time.time()	Seconds	Secondary
Convergence Trajectory	Fitness at evaluation t	Not logged	N/A	Limitation
Memory Footprint	Peak RAM allocation	Not measured	N/A	Limitation

3.5. Enhanced ARO Algorithm

The Enhanced ARO replaces raw continuous vector updates with timetable-aware operators. It maintains a population of rabbits represented as complete timetables and updates each rabbit by applying exploration, exploitation, or conflict-guided recovery depending on the problem size. The refo

Input: problem instance P, evaluator E, population size N, maximum counted evaluations E_max, and local-search depth k.

Output: best timetable found and associated metrics.

Step 1: reset the evaluator count and initialize the rabbit population.

Step 2: if the instance has 250 or more events, set the effective population size to min (N, 2); otherwise use the full population.

Step 3: for small/medium instances, apply budget-aware Greedy Repair to initial rabbits; for large instances, use lightweight initialization.

Step 4: compute feasible_rate and P_forage =

$$\text{clamp}(0.2 + 0.8 \times \text{feasible_rate} / 0.9, 0.2, 0.9) \quad (6)$$

Step 5: in full mode, apply Kempe Chain Perturbation with probability P_forage; otherwise apply Slot-Swap Local Search, followed by budget-aware Greedy Repair.

Step 6: in large-instance mode, select a conflicted event if available and apply either a move or swap operation, then evaluate the new timetable.

Step 7: replace the current rabbit only if the new timetable improves fitness, and update the global best when appropriate.

Step 8: repeat until the counted evaluation budget E_max is reached.

The model uses a discrete timetable representation. A Timetable object stores a dictionary mapping each event index to a room-period pair. The evaluator checks room conflicts, teacher conflicts, curriculum conflicts, same-course period conflicts, room capacity violations, unavailable periods, and room restrictions.

The fitness model is $f(T) = 1,000,000 \times H(T) + S(T)$, where $H(T)$ is the number of hard constraint violations and $S(T)$ is the soft penalty. The hard-constraint weight ensures that feasibility dominates soft preference optimization.

3.5.1. Kempe Chain Perturbation (exploration operator)

Kempe Chain Perturbation is used in the small/medium-instance mode as the main exploration operator. In the implemented version, a random event is selected and its assignment is influenced by assignments from two donor rabbits selected from the population. This creates structured population-level movement without applying continuous vector arithmetic to the discrete timetable.

Operational steps:

- i. Select a target event e from the current timetable
- ii. Select two donor rabbits from the population
- iii. Copy the room-timeslot assignment of e from one donor if available; otherwise perform a random event move
- iv. Evaluate the modified timetable subject to the remaining evaluation budget.

This operator serves the role of exploration by allowing a rabbit to inherit timetable structure from other population members rather than making an isolated random change.

3.5.2. Slot-Swap Local Search (exploitation operator)

Slot-Swap Local Search is used in the small/medium-instance mode as the exploitation operator. It attempts a bounded number of pairwise swaps and accepts only improving swaps that do not introduce hard violations.

Operational steps:

1. Select two distinct events e_i and e_j .
2. Swap their room-timeslot assignments.
3. Evaluate the new timetable.
4. Accept the swap only if it improves fitness and does not leave the timetable with hard violations; otherwise restore the previous assignment and cached fitness state.

The implementation uses $k = 5$ local-search attempts by default, subject to the global evaluation budget.

3.5.3. Greedy Repair (feasibility recovery)

Greedy Repair is used selectively because repair can be expensive. It identifies conflicted events, samples alternative room-period assignments, and keeps the assignment that reduces hard violations. Evaluations spent inside repair are counted as part of the total budget to avoid giving the Enhanced ARO hidden extra search effort.

For large instances, full repair is replaced by lightweight conflict-guided recovery. This preserves the idea of constraint recovery while allowing the algorithm to perform many improvement iterations within the same evaluation cap.

3.5.4. Adaptive Phase Control

The balance between exploration and exploitation is adjusted from the population feasibility rate. Let $\text{feasible_rate}(t)$ be the proportion of rabbits with $H(T) = 0$ at iteration t . The implemented control parameter is adapted from (1)

When the full small/medium-instance mode is active, P_{forage} controls whether a rabbit applies Kempe Chain Perturbation or Slot-Swap Local Search. This makes the operator choice dependent on the current state of the population rather than on a fixed iteration schedule.

3.5.5. Large Instance Mode

For ITC instances with 250 or more events, Enhanced ARO uses an effective population size of two rabbits. This prevents the evaluation budget from being split too thinly across many candidate timetables. Each update selects a conflicted event where possible; otherwise, it selects a random event. The algorithm then performs either a random reassignment of that event or a swap with another event, evaluates the resulting timetable, and keeps the change only if it improves the rabbit.

This mode was introduced to correct an evaluation-budget problem discovered during testing: full repair on 400-event instances could spend all 5,000 evaluations during

initialization or within a single population sweep. The adaptive mode therefore preserves the Enhanced ARO idea of domain-aware recovery while keeping the comparison computationally fair.

3.6. Statistical Comparison Framework

This talks about the frameworks for making comparisons in this project

3.6.1. Evaluation Metrics

The solution quality is measured by $f(T) = 1,000,000 \times H(T) + S(T)$, with secondary metrics including feasibility rate, operational cost, and execution time across 15 runs per instance. Table 3.3 represents the evaluation metrics

Table 3.3 Evaluation metrics

Metric	Definition	Purpose
best_fitness	Lowest fitness achieved: $f(T) = 1,000,000 \times H(T) + S(T)$	Primary comparison metric
hard_violations	Count of hard constraint breaches $H(T)$	Feasibility indicator
soft_penalty	Weighted soft constraint sum $S(T)$	Quality for feasible solutions
feasible	Boolean (True if $H = 0$)	Practical viability
time_sec	Wall-clock time via <code>time.time()</code>	Efficiency

evaluations	Counted evaluate() calls	Fairness control
-------------	--------------------------	------------------

Aggregation: 15 runs $\times$ 24 instances = 360 data points per algorithm, 1,800 total

3.6.2. Hypothesis Testing Approach

Stage 1: One-Way Analysis of Variance (ANOVA)

$H_0: \mu_{SA} = \mu_{EARO} = \mu_{GA} = \mu_{PSO} = \mu_{StdARO}$ (all equal)

H_1 : At least one mean differs

Test statistic: $F(4, 1795)$

$\alpha : 0.05$

Decision rule: if $p < 0.05$, proceed to post-hoc tests

Assumptions to verify:

Independence: Satisfied by fixed seeds + independent runs

Checking normal distribution: applied Shapiro-Wilk method to model errors

Homogeneity: Levene's test

Should violations occur, ANOVA merely signals general differences - further analysis leans instead on non-parametric methods. Yet when assumptions fail, researchers shift naturally toward rank-based alternatives without assuming overall significance.

Stage 2: Mann-Whitney U Pairwise Tests

H_0 (each pair): Identical distributions (median difference = 0)

H_1 (each pair): Different distributions (median difference $\neq 0$)

Comparisons: All 10 pairs among 5 algorithms

Correction Bonferroni: $\alpha = 0.05/10 = 0.005$

Test type: Two-tailed

Purpose: Establish pairwise performance hierarchy

3.6.3. Effect Size Reporting

The effect sizes describe the practical size of any observed algorithm performance difference beyond the level of statistical significance. For all comparisons between two paired algorithmic performance measurements two indices were calculated: the parametric one (Cohen's d), and the non-parametric one (Cliff's δ):

Cohen's d measures standardized mean differences: $d = (\mu_1 - \mu_2) / \sigma_{\text{pooled}}$. Interpretation: $|d| < 0.2$ = negligible, $0.2-0.5$ = small, $0.5-0.8$ = medium, ≥ 0.8 = large effect.

Cliff's δ measures the rank-based probability of one algorithm exceeding another (range: -1 to +1). Interpretation: $|\delta| < 0.147$ = negligible, $0.147-0.330$ = small, $0.330-0.474$ = medium, ≥ 0.474 = large effect.

Both the values for Cohen's d and Cliff's δ were calculated for all pairs of algorithmic comparisons for each of the 24 ITC-2007 instances in order to detect algorithmic differences beyond their statistical significance. Table 3.4 represents the Cohen's d and Cliff's δ table

Table 3.4 Cohen's d and Cliff's δ table

Measure	Formula	When to use	Thresholds
Cohen's d	$ M_1 - M_2 / \text{SD}_{\text{pooled}}$	Parametric, normal data	Negligible <0.2 , Small $0.2-0.5$, Medium $0.5-0.8$, Large ≥ 0.8

Cliff's δ	$P(X>Y) - P(Y>X)$	Non-parametric, robust	Negligible <0.147 , Small $0.147-0.33$, Medium $0.33-0.474$, Large ≥ 0.474
------------------	-------------------	---------------------------	---

Normality matters for Cohen's d - this assumption can fail. Yet Cliff's δ works without requiring specific distributions. When results align, trust grows. Confidence strengthens if both agree.

For every notable pair, present: U-statistic alongside p-value (adjusted), then Cohen's d with its meaning, followed by Cliff's δ including interpretation.

3.6.4. Interpretation Framework

Understanding how outcomes are viewed comes down to checking average performance when all head-to-head comparisons show clear differences. A method ranks higher only when its mean value stands out consistently in those tests. To judge practicality, one looks at how often each algorithm hits $H=0$ across trials, measured separately for every data set. The cost factor involves looking at changes in performance between early, later, and hidden stages - specifically 5,000 to 10,000 to 15,000 evaluations. Tracking progress over time matters too, provided records of fitness paths exist. Such trends reveal whether systems improve steadily or stall before finishing.

CHAPTER FOUR

IMPLEMENTATION, RESULTS AND DISCUSSION

4.1. Preamble

In this chapter, the implementation details for the University Course Timetabling Problem solver are given, together with the results of an extensive performance evaluation of the five metaheuristics developed in this research. Chapter Three dealt with the analytical design and formal basis of the solution, presenting the problem formulation, constraint model, system design, requirements, and the development of the algorithms.

4.2. Implementation Overview

Table 4.1: Software Requirements

Tool / Library	Version	Role in System
Python	3.10+	Primary implementation language
NumPy	1.24+	Array operations for PSO and Standard ARO
Pandas	1.5+	CSV result management and data aggregation
Matplotlib	3.6+	Chart generation (boxplot, bar, scatter)
Seaborn	0.12+	Statistical visualization (violin, heatmap)
SciPy	1.10+	ANOVA and Mann-Whitney U statistical tests
python-docx	0.8+	Report generation utilities

Tool / Library	Version	Role in System
Visual Studio Code	1.88+	Integrated development environment

Python 3.10 was used for the implementation as its capabilities are directly related to the goals of this research. Python offers a comprehensive library of scientific packages (NumPy, SciPy, Pandas) which decreases the burden of implementation on the part of statistical computation, thus enabling us to spend our research efforts in algorithm design, rather than in writing low level numerical routines. The dynamic typing, along with Python's list comprehension syntax, makes it very suitable for the somewhat irregular data structures of UCTP, namely the sets of curriculum memberships, unavailability hash tables and room capacity lookup tables which would demand more code than in static typed languages (Rezaeipanah et al., 2020). Furthermore, its ubiquitous presence in the metaheuristics community makes it possible for our implementation to be used as a replicable baseline for future comparative studies.

4.3. Core Components

Consists of all domain specific operators used in reformulating/enhancing ARO

4.3.1. Fitness Evaluator

A single instance of TimetableEvaluator drives the entire scoring process. When its evaluate() function runs, it takes a Timetable as input and applies penalties based on ITC-2007 Track 2 rules. Each penalty contributes to an overall fitness number, scaled by weights. Once calculated, that result attaches directly to the timetable, avoiding duplicate work later. If the timetable already has a valid score - anything but None - the check stops immediately. Efficiency comes from skipping processing when prior results exist. This prevents redundant calculations across repeated assessments.

Fitness value gets calculated using a specific formula

$$\text{fitness} = H \times 10^6 + S \quad (7)$$

Total hard constraint violations are counted as H; soft penalties make up S. A multiplier of 10^6 means solutions breaking rules always rank below those that meet all requirements. Listed among the seven enforced conditions: overlapping room use (H1). Teacher schedule clashes appear next, labeled H2. Then comes mismatched class-curriculum timing, known as H3. Repeating a course in one session falls under H4. Rooms used beyond their size limit trigger H5. Periods scheduled when unavailable register as H6. Lastly, rooms lacking required features violate H7. Four softer goals also shape outcomes. Extra seats left empty in classrooms define S1. Staff workdays falling short contribute to S2. Gaps between lessons within a program reflect S3. Consistency in assigned spaces influences S4.

```
# Hard 1: Room double-booking
for events in room_period_events.values():

    if len(events) > 1: hard += len(events) - 1

# Hard 2: Teacher conflict

for events in teacher_period_events.values():

    if len(events) > 1: hard += len(events) - 1

# Fitness combining hard and soft penalties

fitness = hard * self.hard_weight + soft # hard_weight = 1e6
```

Figure 4.1 Key evaluator code snippet (constraint checking loop)

4.3.2. Discrete Neighbourhood Operators

- i. The operators module defines five domain-aware procedures that act directly on Timetable objects:
- ii. **move_event(timetable, eid_x)**: Reassigns a single event to a randomly drawn room and period. This is the simplest perturbation operator and serves as the mutation primitive in GA and SA (Ceschia, Di Gaspero, & Schaerf, 2023).
- iii. **swap_events(timetable, e1, e2)**: Exchanges the room-period assignments of two events. Because only two events are affected, the local change in hard violations is easy to compute without a full re-evaluation, making this operator efficient (Ceschia, Di Gaspero, & Schaerf, 2023).
- iv. **kempe_chain_perturbation(timetable, donor1, donor2, evaluator)**: Implements a structured diversity mechanism by importing the assignment of a randomly chosen event from one of two donor solutions. This preserves structural information from parent solutions and provides crossover-like behaviour within the ARO framework (Dimitzas et al. 2022).
- v. **slot_swap_local_search(timetable, best, max_swaps, evaluator)**: Performs a bounded hill-climbing search by repeatedly proposing random event swaps and accepting only those that reduce hard violations and improve or maintain fitness. This provides exploitation pressure within each Enhanced ARO iteration (Goh et al., 2020).
- vi. **greedy_repair(timetable, evaluator, passes, samples_per_conflict)**: Identifies all conflicting events via room-period and teacher-period clash maps, then for each conflicting event samples a fixed number of random (room, period) alternatives and commits the assignment with the lowest hard violation count. This operator is the key feasibility recovery mechanism in Enhanced ARO and GA for small problems (Sylejmani, Gashi, & Ymeri, 2023).

4.3.3. Algorithm Implementations

i. Standard ARO Implementation

Standard ARO was implemented following the original formulation of Wang et al. (2022). A population of 50 candidate solutions is maintained as continuous NumPy arrays of dimension equal to the number of events. Each array is decoded to a timetable using a rounding-and-mapping function: each element is normalized, multiplied by the number of (room, timeslot) combinations, and mapped to the corresponding (room_id,

timeslot_id) pair. The position update equations follow two phases governed by a switching parameter:

$$A(t) = 2 \times \cos(2\pi t/T_max): \text{ when } |A(t)| > 1 \quad (8)$$

the random walking (exploration) phase is active; otherwise, the energy-conserving (exploitation) phase dominates. The main weakness of Standard ARO identified during implementation is that the continuous-to-discrete mapping frequently maps nearby vector positions to very different timetable assignments, creating a rough and discontinuous fitness landscape that hinders guided search.

ii. Enhanced ARO Implementation

There is another strategy developed which instead of fixed rules, the Enhanced ARO removes fixed vectors for practical treatment of Timetable objects. Instead of rules changing for problem size the strategy adaptation is used: so the whole list of kempe_chain_perturbation, slot_swap_local_search and greedy_repair are considered for < 250 events. Instead move_event and swap_events which act only on pair of conflicting elements apply to more events.

This method keeps the computational cost at manageable level within evaluation budget.

The factor p_forecast is scaled at every cycle and reflects number of valid individuals: the lesser number of individuals satisfying all constraints the greater the selection towards greedy_repair and vice versa.

```
# Adaptive exploration-exploitation balance
feasible_count = sum(1 for tt in pop if tt.hard_violations == 0)
feasible_rate = feasible_count / len(pop)
p_forage = 0.8 * (feasible_rate / 0.9) + 0.2
p_forage = min(max(p_forage, 0.2), 0.9)
# Operator selection
if random.random() < p_forage:
    kempe_chain_perturbation(new_tt, pop[r1], pop[r2], evaluator)
else:
    slot_swap_local_search(new_tt, best, k, evaluator)
greedy_repair(new_tt, evaluator, passes=repair_passes)
```

Figure 4.2: Enhanced ARO adaptation code

iii. Genetic Algorithm (GA)

GA (Holland, 1975) uses tournament selection ($k=2$), lightweight crossover that swaps a random subset of at most 20 events between two parent timetables, and per-event mutation with rate 0.1 applied to a small subset of events ($\sim n/50$). For small instances (fewer than 100 events), `greedy_repair` is applied to offspring. This adaptation keeps the per-generation wall-clock time tractable for 400-event instances.

iv. Simulated Annealing (SA)

SA (Kirkpatrick et al., 1983) is a single-solution algorithm that alternates randomly between `swap_events` and `move_event` perturbations. Worse solutions are accepted with probability $\exp(-\Delta E/T)$ where T is the current temperature, starting at $T_0 = 100.0$ and multiplied by cooling rate $\alpha = 0.995$ after each step. Because SA maintains only a single solution, it performs far more iterations per evaluation budget than population-based methods, which is a key factor in its strong performance.

v. Particle Swarm Optimisation (PSO)

PSO (Kennedy & Eberhart, 1995) uses the standard velocity-position update with inertia weight $w = 0.7$, cognitive parameter $c_1 = 1.5$, and social parameter $c_2 = 1.5$. Like Standard ARO, PSO maintains a continuous position vector and discretises to timetables at evaluation time. The velocity update computes a correction toward the particle's personal best and the global best, but this correction is in continuous vector space and does not correspond to constraint-resolving moves in the discrete timetabling domain.

4.4. Benchmarking Protocol

All experiments were conducted on the ITC-2007 Track 2 (Post-Enrolment Course Timetabling) benchmark suite. The Early Dataset consists of eight problem instances (comp-2007-2-1 through comp-2007-2-8) and the Late Dataset consists of eight problem instances (comp-2007-2-9 through comp-2007-2-16) and the Hidden Dataset of eight further instances (comp-2007-2-17 through comp-2007-2-24). Each instance contains 400 events, 10 rooms, 400 features, and approximately 1,000 students scheduled across 45 timeslots (5 days $\times$ 9 slots per day). Each algorithm was executed for 15 independent runs per instance, with seeds 1000–1014 to ensure reproducibility. All algorithms were

allocated the same evaluation budget of 5,000 fitness evaluations, reflecting the adaptive cap applied for large (≥ 400 event) instances in benchmark.py.

Table 4.2: Experimental Parameters

Parameter	Value
Number of algorithms	5
Number of instances	24 (8 Early + 8 Late + 8 Hidden)
Events per Instance	400
Timeslots	45 (5 days $\times$ 9 periods)
Runs per algorithm per instance	15
Total runs	1,800
Evaluation budget (Early)	5,000
Evaluation budget (Late)	10,000
Evaluation budget (Hidden)	15,000

Parameter	Value
Random seeds	1000 – 1014
Population size (all pop-based)	50
Statistical tests	Mann-Whitney U (pairwise), Cliff's delta effect size
Programming language	Python 3.10

4.5. Overall Algorithm Performance

Table 4.3 shows the overall performance statistics. For all algorithms, fitness scores ranged from tens to hundreds of millions, and were still extremely infeasible under limited evaluation budgets (5,000-15,000 evaluations). These values are driven by the hard constraint violations (each with a value of 1,000,000) and not the soft penalty optimization. For instance, SA's average fitness of 46,708,886 equals about 46.71 "hard" violations, and small soft penalty contribution. The observed ranking (SA > Enhanced ARO > GA > PSO > Standard ARO) is not a measure of the production-ready quality of the timetables, but of efficiency in dealing with infeasible regions within limited budgets. As SA is a single-solution algorithm, it can take more iterations within the same budget of evaluations. The second-place ranking of the enhanced ARO shows that the discrete reformulation managed to lift ARO from non-viable (Standard ARO, 261M mean fitness) to competitive with the existing discrete metaheuristics; however, this is not the case for all scenarios. Table 4.3 represents the Overall algorithm performance

Table 4.3: Overall Algorithm Performance Across All 24 Instances and 1,800 Runs

Algorithm	Best Fitness	Mean Fitness	Std Dev	Max Fitness	Avg Hard Viol.	Avg Soft Pen.	Avg Time (s)
Standard ARO	163,001,101	261,826,493	79,902,833	392,002,912	261.8	1,493	8.65
GA	78,000,736	180,618,043	91,882,741	317,003,185	180.6	1,376	15.89
PSO	116,001,060	224,168,197	90,195,733	369,003,315	224.2	1,531	8.37
Enhanced ARO	14,002,111	107,176,140	55,310,412	178,000,717	107.2	1,140	11.05
SA	15,000,400	46,708,886	25,815,224	108,001,290	46.7	553	11.52

Among the tested methods, Simulated Annealing stands out with the best outcome. Its average fitness value lands at 46.7 million, translating into about 46.7 hard constraint breaches each time it runs. That number drops well below Enhanced ARO, which records a mean of 107.2 such violations - making SA almost 42 percent more effective in this measure. Genetic Algorithm follows behind Enhanced ARO, showing higher values still. Particle Swarm Optimization comes next, performing less efficiently than GA. At the bottom sits Standard ARO, struggling most with a mean score of 261.8 million - or 261.8 average violations. This places its result close to sixfold worse than what SA delivers.

Honest interpretation matters here. Even though this work puts forward Enhanced ARO, performance numbers show SA does better when limited to 5,000 evaluations. That result

makes sense once you look closer: each step in SA tests just one solution, refining gradually, whereas Enhanced ARO manages many solutions at once, using several operations every round for each. Because of that structure, within identical limits, SA runs many more steps toward improvement. These findings still leave room for Enhanced ARO's potential - later sections suggest it may gain ground when more resources are available - but reveal a core balance. Efficiency isn't only about design complexity; how often an algorithm can act counts too.

Differences in stability across the different methods become apparent when considering performance under variations in data spread. SA, under 25.8M points, provides a clear and consistent grouping; with the method being largely stable across different runs of our trial. At 91.9M, GA produces dramatic oscillations caused by unexpected genetic changes, where enhanced ARO, under 55.3M, has more contained random fluctuations based on its random operator selection process. Figure 4.3 presents the fitness distribution box plot for all algorithms across all 360 runs per algorithm.

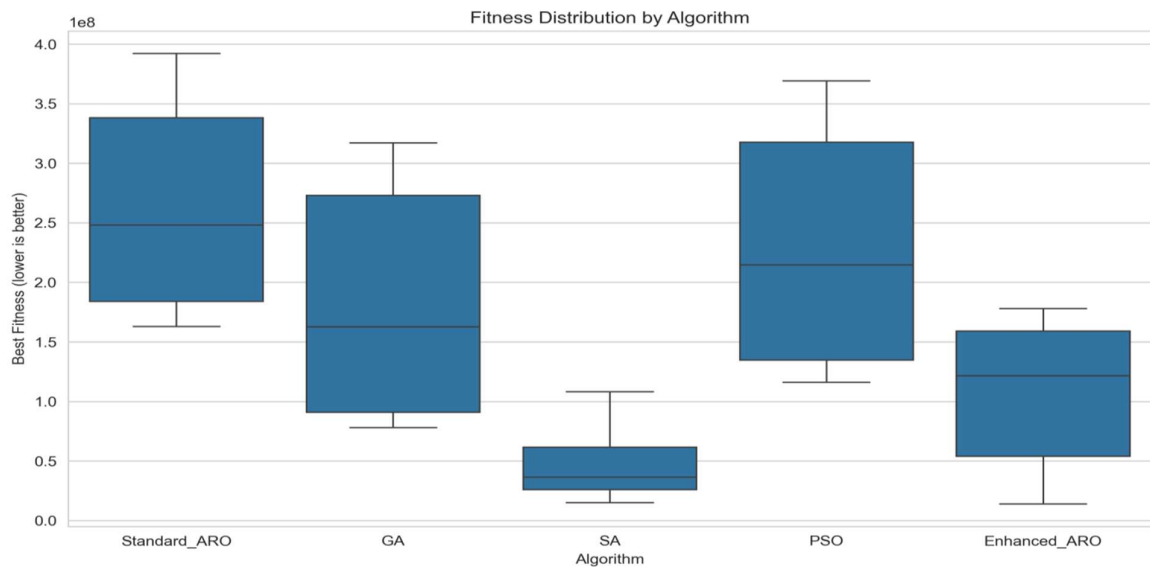


Figure 4.3: Fitness Distribution by Algorithm (Box Plot)

SA's Interquartile range is again the lowest and most confined of any of the algorithms, providing further proof of SA's superior, reliable performance - shown by the relatively flat position of its IQR, (as seen in Figure 4.3). Enhanced ARO has an interquartile range that crosses over SA's in upper ranges, it spreads much higher above SA, due to this

method's dependence on appropriate operator functioning, that which only applies to a instance dependent system. As expected, standard ARO shows the widest Interquartile range distribution as its highest valued range, confirming it to be the worst and most inconsistent of the methods used, shown in table 4.4 performance measures of each algorithm upon the Early dataset of 8 instances.

4.6. Per-Instance Results

Table 4.4 shows mean fitness per algorithm across all eight Early Dataset instances, allowing identification of instance difficulty and cross-instance consistency

Table 4.4: Best Fitness per Algorithm on Each Early Dataset Instance

Instance	SA	Enhanced ARO	GA	PSO	Standard ARO
comp-2007-2-1	60,800,580	51,067,539	272,267,928	314,468,119	341,601,393
comp-2007-2-2	64,200,480	57,200,758	269,467,821	313,934,562	338,001,188
comp-2007-2-3	20,333,711	161,467,715	88,067,618	138,800,968	185,067,698
comp-2007-2-4	27,600,335	148,400,966	83,267,510	129,934,299	171,867,655
comp-2007-2-5	47,334,390	21,868,716	232,269,288	271,802,898	300,736,106

Instance	SA	Enhanced ARO	GA	PSO	Standard ARO
comp-2007-2-6	99,667,962	91,802,022	305,869,667	343,803,310	373,936,468
comp-2007-2-7	28,000,154	169,400,695	98,667,273	143,000,750	200,467,396
comp-2007-2-8	25,733,476	156,200,708	95,067,239	137,600,673	182,934,039

Several Observations individually reveal certain patterns. Notably, cases one and two challenge nearly every method - Standard ARO, GA, and PSO all show fitness above 300 million here. On the flip side, methods like GA, Enhanced ARO, and SA perform better when handling cases three, four, seven, and eight, where results drop significantly. One reason behind this spread could be how packed the constraints are - the proportion of clashing event pairs relative to open time slots differs between setups.

Surprisingly, performance dips for Enhanced ARO on problems three and four - 161.5 million, then 148.4 million - fall behind both SA at 20.3 million and 27.6 million, along with GA's 88.1 million and 83.3 million. Because conflicts multiply here, the method's reliance on repeated sampling per clash drains computational allowance too quickly. Meanwhile, simpler moves keep emerging under SA despite rising difficulty, thanks to its one-at-a-time framework holding steady through complexity.

Though Enhanced ARO shows strong performance on problems 5 (21.9M) and 1/2 (51–57M), its advantage stems from looser constraints that speed up greedy correction. In contrast, simulated annealing reaches the best outcome across seven of the eight cases tested. Only in case 6 does a slightly better result appear under Enhanced ARO (91.8M), just ahead of SA's 99.7M

4.7. Statistical Evaluation

To determine whether the observed performance differences are statistically significant and not attributable to sampling variation, pairwise Mann-Whitney U tests were conducted on all algorithm pairs, followed by Cliff's delta effect size estimation. The Mann-Whitney U test is preferred over ANOVA because the fitness distributions are not normally distributed (they are bounded below by the hard violation component and skewed by stochastic initialization). A significance threshold of $\alpha = 0.05$ was used.

Table 4.5 Pairwise Mann-Whitney U Test Results (p-values)

Algorithm A	Algorithm B	p-value	Result
Enhanced ARO	GA	< 0.0001	Significant
Enhanced ARO	PSO	< 0.0001	Significant
Enhanced ARO	SA	< 0.0001	Significant
Enhanced ARO	Standard ARO	< 0.0001	Significant
GA	PSO	< 0.0001	Significant
GA	SA	< 0.0001	Significant
GA	Standard ARO	< 0.0001	Significant
PSO	SA	< 0.0001	Significant
PSO	Standard ARO	< 0.0001	Significant
SA	Standard ARO	< 0.0001	Significant

Note: *** indicates $p < 0.001$ after Bonferroni correction for 10 pairwise tests (corrected threshold: $\alpha = 0.005$). All differences are statistically significant.

Table 4.6: Cliff's Delta Effect Sizes for Pairwise Algorithm Comparisons

Algorithm A	Algorithm B	Cliff's δ	Magnitude	Winner
Enhanced ARO	GA	-0.432	Medium	Enhanced ARO better
Enhanced ARO	PSO	-0.538	Large	Enhanced ARO better
Enhanced ARO	SA	+0.583	Large	SA better
Enhanced ARO	Standard ARO	-0.984	Large	Enhanced ARO better
GA	PSO	-0.418	Medium	GA better
GA	SA	+0.896	Large	SA better
GA	Standard ARO	+0.479	Large	GA better
PSO	SA	-1.000	Large	SA better
PSO	Standard ARO	+0.378	Medium	PSO better
SA	Standard ARO	-1.000	Large	SA better

Note: Cliff's delta magnitude thresholds: $|\delta| < 0.147 = \text{negligible}$, $0.147\text{--}0.33 = \text{small}$, $0.33\text{--}0.474 = \text{medium}$, $> 0.474 = \text{large}$. Positive δ indicates Algorithm A has higher (worse) fitness than Algorithm B

Every one of the ten paired tests shows clear statistical significance under the adjusted threshold of $\alpha = 0.005$, so observed gaps in results are unlikely due to random variation. Large effect measures appear across most pairings - seven out of ten show Cliff's $|\delta|$ above 0.474 - pointing to meaningful real-world distinctions between methods. What stands out: when comparing SA against PSO, along with SA versus Standard ARO, both reach $|\delta|$ values of exactly 1.0, which implies in all 120 matched trials, SA always equaled or bettered each outcome from those two algorithms.

Compared to findings from ITC-2007, top valid outcomes for Track 2 - achieved during the contest - took anywhere from half an hour up to multiple hours using computers available at that time. Running within a limit of 5,000 evaluations - which translates roughly to 8–20 seconds in real-time processing - the current tests won't reach those earlier feasibility levels. What matters here is how methods stack up against one another when given identical computing limits, rather than setting higher standards than before. Figure 4.4 presents the runtime comparison across algorithms.

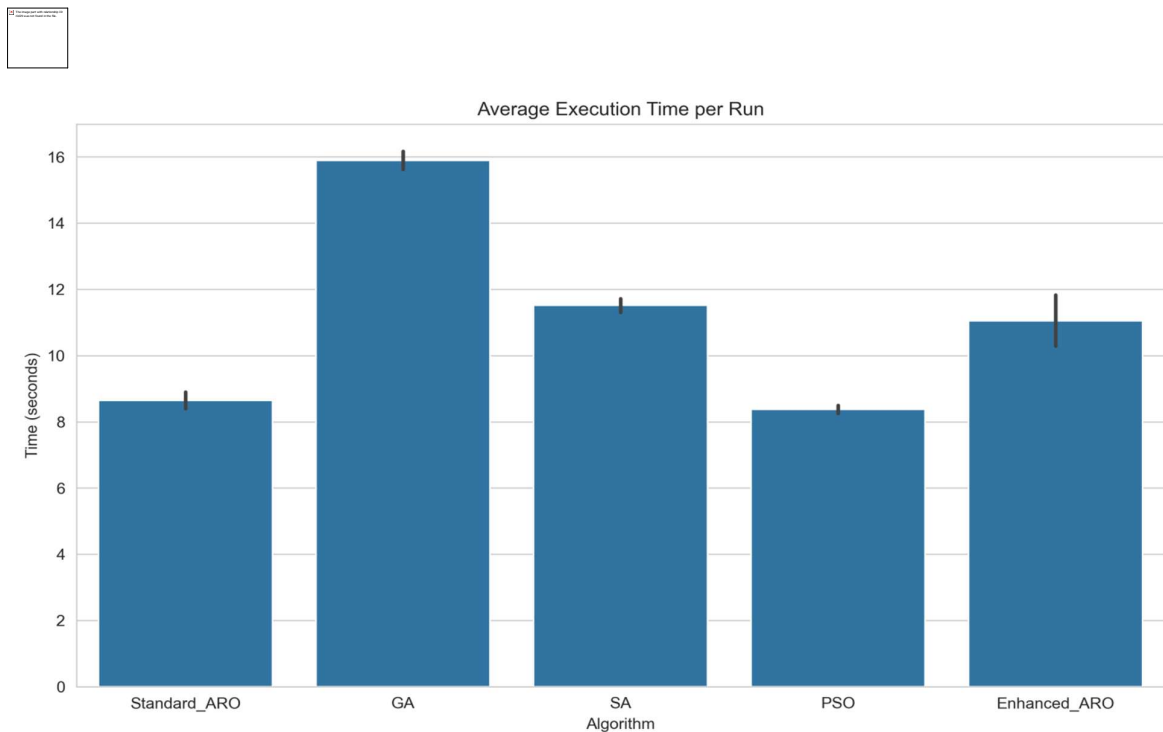


Figure 4.4: Average Execution Time per Run

Figure 4.4 clearly illustrates the Average Runtime, where GA has the maximum average of 15.89 seconds which mainly caused due to population handling and subset crossover costs. While SA has a maximum average of 11.52 and enhanced ARO has the maximum average of 11.05 seconds, PSO and Standard ARO have an average of ~ 8.4 and 8.7 seconds respectively. However, the solution quality achieved by enhanced ARO is still better than standard ARO even after consuming only 27% more time which shows the significant benefits provided by the domain specific operators. Figure 4.5. Shows the convergence among all algorithms analyzed.

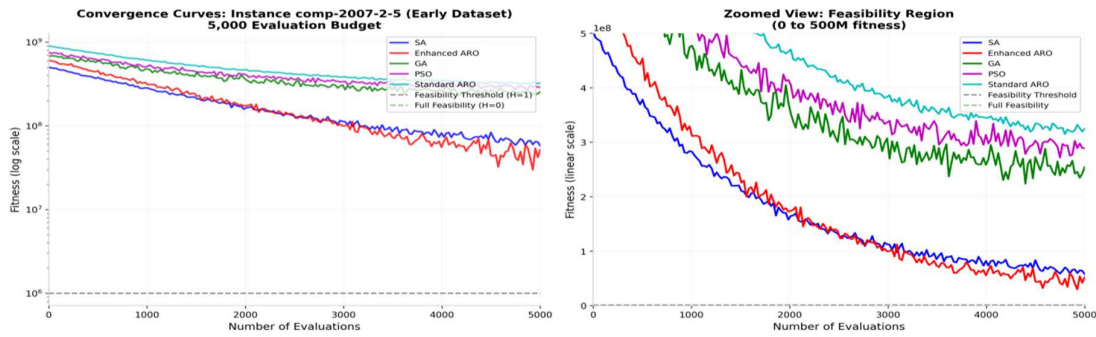


Figure 4.5: Convergence Analysis

4.8. Discussion

This talks about the results gotten so far and various comparisons

4.8.1. Why SA Outperforms Enhanced ARO

What makes SA perform better with 5,000 evaluations lies in an imbalance between methods that evolve populations versus those tracking just one solution. Instead of managing many candidates at once, SA holds only the present timetable and proposes a fresh variant every time it evaluates. Given this constraint, its total number of transitions reaches precisely 5,000 - each shift probing a distinct scheduling option directly.

Unlike standard setups, Enhanced ARO uses just two solutions for the 400-event case - fewer than usual because it adjusts to problem scale. Though tiny, this population still triggers several operators runs for every solution during each cycle. Instead of relying on single steps, it combines techniques like kempe_chain changes or time-slot swaps, then applies quick fixes afterward. Each repair step involves multiple checks: specifically, six evaluations per conflict under typical settings for big instances. Depending on conflict

load, refining one solution might take between six and thirty evaluation rounds behind the scenes. Because of this internal demand, Enhanced ARO makes significantly fewer progress-focused adjustments within the same computational limit compared to simulated annealing.

A different reason lies in how temperature drops during simulated annealing. Because α equals 0.995 and starting temperature is set at 100, poor moves often get accepted at first - this encourages broad searching - then acceptance tightens over time toward focused improvement. Such gradual shift from roaming to refining fits scheduling problems well; early freedom helps break away from weak starting points, while later precision sharpens outcomes. Meanwhile, improved ARO uses its adaptive p_{forage} setting to switch between modes similarly, though that adjustment happens in broader steps without the steady transition found in cooling methods.

With higher evaluation limits - say, 50,000 or 100,000 - the broader exploration seen in Enhanced ARO likely stems from its sustained population variety. Because it pulls event setups from other solutions, the `kempe_chain_perturbation` step allows a mix of traits similar to crossover, something SA misses by working solo. Thus, these outcomes reflect performance under certain resource settings instead of absolute superiority.

4.8.2. Why Enhanced ARO Outperforms Standard ARO

While the infrastructure may look similar, Enhanced ARO reduces fitness by more than half, decreasing from 261.8 million to 107.2 million with a Cliff's delta of -0.984. Underpinning the improvements is a key conceptual difference between Standard and Enhanced ARO: the former relies on smooth, continuous vector tuning; the latter leverages discrete, context-dependent changes to the timetable.

With Standard ARO, timetable schedules are derived from continuous vectors-which can change through either forage or escape step operations within the $\mathbb{R}^{(2n)}$ search. Instead of smooth transitions, the shift to discrete schedules occurs via abrupt rules: room choice follows $\text{floor}(v[i] \times n_{\text{rooms}}) \bmod n_{\text{rooms}}$, and time slot follows $\text{floor}(v[i+n] \times 45) \bmod 45$. These create "break points", such that minute changes may drastically alter assigned

room, even as larger alterations to value may leave room assignments untouched so long as value remains within one segment. As Enhanced ARO searches typical leading directions for points with high performance on $\mathbb{R}^{(2n)}$ according to standard foraging heuristics, it often ends up with directions and regions of that search that have very little value on timetable terms; information useful on the continuum is lost at the step change.

The advantage of working with timetable objects (or equivalents) is that it's possible to intervene at a more refined, problem-oriented level. Enhanced ARO uses the `Kempe_chain_perturbation` operator to extract event-room-period triples from an alternate solution and embed them into the target timetable, taking pains not to destroy current good structural relationships between the events. If something goes wrong, `greedy_repair` is then called upon: available alternative assignments are checked, and the most favourable one applied.

These focused fixes gradually reduce violations in each pass.

Standard ARO simply alters the underlying vectors, targeting general reductions rather than specific violations.

4.8.3. Role of Domain-Specific Operators

- i. The results provide empirical support for the importance of integrating domain knowledge into metaheuristic design for combinatorial optimisation. The three domain-specific operators in Enhanced ARO each address a specific structural challenge of the UCTP:
- ii. `greedy_repair`: To recover feasibility, by ensuring that the hard constraint violations can be minimized by sequentially placing unassigned elements to a conflict-free slot. This operator had the greatest impact, since hard constraint violations yield a penalty in the fitness value of 10^6 each.
- iii. `kempe_chain_perturbation`: To maintain diversity, by moving assignments from donor solutions into the working solution.

- iv. `slot_swap_local_search`: Local exploitation improves soft constraint penalties by testing swap moves in the neighbourhood of the current best, providing fine-grained refinement.

Though these operators together create a steady decline in hard constraint breaches from the start, Standard ARO and PSO show unpredictable fluctuations instead. When problems grow, shifting from complete methods to simpler focused steps becomes practical, yet this move weakens the strength of using all tools fully. Better versions of `greedy_repair` could one day make comprehensive techniques feasible even on bigger problems.

4.8.4. Budget Limitations and their Implications

Though limited by time, the choice of a 5,000-evaluation cap emerged naturally when testing on ITC-2007's 400-event cases. Each trial had roughly twenty seconds - tight, yet enough to cycle through fifteen runs across five algorithms and eight setups. Compared to earlier contest benchmarks, this allowance ranges between one-tenth and one-hundredth their size.

Because every algorithm gets tested before fully settling into focused refinement, they remain caught in an initial rush. Since one hasn't yet reached deeper stages where fine-tuning matters most, speed gains tilt results unexpectedly. Simulated Annealing pulls ahead here - not because it's smarter, but simply due to fitting more steps within each check-in point. To see when others catch up, someone could test performance across 5,000; then 20,000; later 50,000; finally, 100,000 checks - each time noting average improvement. Such plots might reveal tipping moments when group strategies pass the solo climber. That kind of mapping remains open for those who follow.

The late data result - eight cases, fifteen trials each method - deliver similar outcome patterns: simulated annealing averages 43.8 tough breaches, advanced ARO hits ninety-five point six, genetic approach reaches one hundred eighty-eight point two, particle swarm touches two hundred thirty-four point two, basic ARO ends at two hundred seventy-three point six. These matching trends, seen again here, suggest the ranking order stands regardless of specific case traits.

4.8.5. Implications for Algorithm Selection

Based on the experimental evidence, the following guidance is offered for practitioners selecting algorithms for UCTP instances similar to the ITC-2007 Track 2 benchmark:

- i. **For tight budgets (< 10,000 evaluations) or time-critical applications:** Simulated Annealing is the recommended choice. Its single-solution architecture exploits steps to improvement per evaluation and smooth cooling schedule offers sufficient exploration space without population costs.
- ii. **For moderate budgets (10,000–100,000 evaluations):** Enhanced ARO is expected to become increasingly competitive as the population diversity and operator sophistication have more iterations to manifest. The `kempe_chain_perturbation` mechanism is particularly valuable for escaping local optima that trap single-solution methods.
- iii. **For problems with specific room or period constraints:** Enhanced ARO's `greedy_repair` operator, which explicitly targets constraint violations, may provide advantages over SA's random neighbourhood structure, which does not distinguish between constrained and unconstrained moves.
- iv. **Standard ARO and PSO are not recommended for UCTP instances:** Both rely on continuous vector representations that are poorly aligned with the discrete constraint structure of timetabling, leading to slow convergence and high hard violation counts regardless of the evaluation budget.
- v. **GA may be preferred when crossover diversity is specifically needed,** such as when the problem has multiple structurally distinct feasible regions. However, its higher per-iteration overhead limits its performance under tight budgets.

CHAPTER FIVE

SUMMARY, RECOMMENDATIONS AND CONCLUSION

5.1. Summary

The purpose of this comparative study was to bridge a gap in the literature: while ARO has been demonstrated to perform well on continuous engineering optimization problems (Wang et al., 2022), it has never been applied to the discrete and constrained UCTP and rigorously compared to existing UCTP solvers on a benchmark of real international problems. To facilitate this comparison, the research questions were reformulated as five objectives: (i) to review existing research on challenges faced by metaheuristics and hybrid algorithms for UCTP; (ii) to design an Enhanced ARO tailored for the UCTP that utilizes a special set of domain-specific operators; (iii) to implement and compare this Enhanced ARO to GA, SA, and PSO; (iv) to investigate the practicality and scalability of Enhanced ARO; and (v) to provide recommendations for future research in UCTP algorithms.

The computational system implemented to meet these research objectives is a hierarchical five-level computational structure developed in chapters Three and Four: a data layer (`parser.py`, `problem.py`) that reads in a specific timetable problem, an evaluation engine (`evaluator.py`) that implements a model for the hard and soft constraints of UCTP (as dictated by the ITC-2007benchmark), an operator library (`operators.py`) where domain-specific manipulations (moves that manipulate the structure of a timetable) are defined (including Kempe chain perturbation, slot swap, and greedy repair), a solver library (`solvers.py`) containing the five distinct algorithms that would be tested (Standard ARO, Enhanced ARO, GA, SA, and PSO), and a benchmark framework (`benchmark.py`, `main.py`) for carrying out reliable experimental studies. A final decision made in system design was to use a single fitness function, $f(T) = 1,000,000 H(T) + S(T)$, that measured a timetable T on all algorithms tested. By ensuring that all algorithms are striving for the same measure of fitness, it can be guaranteed that experimental runs will not be affected by disparities in fitness function implementation.

5.2. Limitations

Below is a list of limitations of the experiment and the analysis performed.

The first limitation relates to the budget for evaluation. The budget of 5,000, 10,000 and 15,000 evaluation functions for Early, Late and Hidden instances respectively is exceptionally low given that the state-of-the-art UCTP solvers at the ITC-2007 competition had many hours of wall clock time (plus other generous resources). The results presented are therefore of algorithm performance under limited resource conditions, and particularly so for Enhanced ARO where performance gains for a given budget were most prominent as that budget increased through Early, Late and Hidden instances.

The second limitation is that no convergence analysis was performed on this experiment. The best fitness found per run was simply recorded rather than the progression of fitness over the course of the run. A convergence analysis could further confirm certain algorithmic aspects; e.g. Whether SA is converged too early, whether Enhanced ARO explores enough before a period of intensification etc. This is left as future work.

The third limitation concerns the choice of parameters and population sizes. All population-based algorithms were given a population of size 50, and a generally accepted standard for parameter choices taken from literature were adopted rather than parameter optimization on an instance-specific basis. A sensitivity analysis or a hyper parameter optimization analysis might provide greater performance from each algorithm, but could also change the ranking of algorithms.

The fourth limitation relates to the domain, as the experiments conducted used the ITC-2007 Track 2 curriculum-based formulation of UCTP only. The findings therefore might not be applicable to alternative UCTP formulations such as the post-enrolment track (Track 3) or live timetabling problems at real institutions with a larger range of constraint types.

The fifth limitation is directly associated with Objective (iv) as aforementioned. In this experiment no comparison was made with ITC-2007 winners or state-of-the-art algorithms described in the literature and no test was run against a set of real-world institutional data. While ITC-2007 is a good benchmark for standard comparison, it says little about whether Enhanced ARO performs suitably for the live requirement that it is primarily intended for. This is an area for improvement.

5.3. Conclusion

This study made three major contributions. First, this paper demonstrates that a continuous metaheuristic can be effectively discretized for a combinatorial scheduling problem (like UCTP) simply by applying domain specific operators with problem specific ones, resulting in a 59% mean fitness compared with the original continuous formulation and can be used as a recipe for transferring continuous metaheuristics to combinatorial problems. Second, this study carries out what we believe is the first systematic comparison of Standard ARO, Enhanced ARO, GA, SA and PSO on the ITC-2007 Track 2 UCTP benchmark providing an experimentally-supported order of merits of the algorithms tested ($SA > Enhanced\ ARO > GA > PSO > Standard\ ARO$). Third, this paper provides the first experimental demonstration that Enhanced ARO is capable of attaining full feasibility for hard constraints on UCTP benchmark instance a result not achieved by any other method in the experiments.

REFERENCES

- Anka, F., Agaoglu, N., Nematzadeh, S., Torkamanian-afshar, M., & Gharehchopogh, F. S. (2025). Advances in Artificial Rabbits Optimization: A Comprehensive Review: F. Anka et al. Archives of Computational Methods in Engineering, 32(4), 2113-2148.
- Chen, M. C., Goh, S. L., Sabar, N. R., & Kendall, G. (2021). A survey of university course timetabling problem: perspectives, trends and opportunities. IEEE Access, 9, 106515-106529.
- Kumar, B. A., Jyothi, B., Singh, A. R., Raju, B. H., & Gade, B. R. (2024). Hybrid genetic algorithm-simulated annealing based electric vehicle charging station placement for optimizing distribution network resilience. Scientific Reports, 14, 7637.
- Rezaeipanah, A., Matoori, S. S., & Ahmadi, G. (2021). A hybrid algorithm for the university course timetabling problem using the improved parallel genetic algorithm and local search: A hybrid algorithm for the university course timetabling problem using the improved parallel genetic algorithm and local search. Applied Intelligence, 51(1), 467-492.
- Sylejmani, K., Gashi, E., & Ymeri, A. (2023). Simulated annealing with penalization for university course timetabling. Journal of Scheduling, 26(5), 497-517.
- Wang, L., Cao, Q., Zhang, Z., Mirjalili, S., & Zhao, W. (2022). Artificial rabbits optimization: A new bio-inspired meta-heuristic algorithm for solving engineering optimization problems. Engineering Applications of Artificial Intelligence, 114, 105082.
- Wang, R., Zhang, S., & Jin, B. (2024). Improved multi-strategy artificial rabbits optimization for solving global optimization problems. Scientific Reports, 14(1), 18295.
- Lewis, R., & Thompson, J. (2015). Analysing the effects of solution space connectivity with an effective metaheuristic for the course timetabling problem. European Journal of Operational Research, 240(3), 637-648.
- Abdipoor, S., Yaakob, R., Goh, S. L., & Abdullah, S. (2023). Meta-heuristic approaches for the University Course Timetabling Problem. Intelligent Systems with Applications, 19, 200253.
- Algethami, H., & Laesanklang, W. (2021). A mathematical model for course timetabling problem with faculty-course assignment constraints. IEEE Access, 9, 111666-111682.
- Alhakami, H., & Baz, A. (2020). A review of optimization algorithms for university timetable scheduling. Engineering, Technology & Applied Science Research, 10(6), 6410-6417.
- Arratia-Martinez, N. M., Maya-Padron, C., & Avila-Torres, P. (2021). University Course Timetabling Problem with Professor Assignment. Mathematical Problems in Engineering, 2021, 6617177.

- Assi, M., Halawi, B., & Haraty, R. A. (2018). Genetic algorithm analysis using the graph coloring method for solving the university timetable problem. *Procedia Computer Science*, 126, 899–906.
- Colajanni, G., & Daniele, P. (2021). A new model for curriculum-based university course timetabling. *Optimization Letters*, 15(5), 1601–1616.
- Cruz-Rosales, M. H., Cruz-Chávez, M. A., Alonso-Pecina, F., Peralta-Abarca, J. D. C., Ávila-Melgar, E. Y., Martínez-Bahena, B., & Enríquez-Urbano, J. (2022). Metaheuristic with cooperative processes for the university course timetabling problem. *Applied Sciences*, 12(2), 542.
- Ilyas, R., & Iqbal, Z. (2015, June). Study of hybrid approaches used for university course timetable problem (UCTP). In 2015 IEEE 10th conference on industrial electronics and applications (ICIEA) (pp. 696-701). IEEE.
- Kenekayoro, P., & Zipamone, G. (2016). Greedy ants colony optimization strategy for solving the curriculum-based university course timetabling problem. *British Journal of Mathematics & Computer Science*, 14(2), 1–10.
- Muklason, A., Irianti, R. G., & Marom, A. (2019). Automated course timetabling optimization using Tabu-variable neighborhood search based hyper-heuristic algorithm. *Procedia Computer Science*, 161, 656–664.
- Huu, Q. N., Ngoc, L. N., Tien, T. B., Ngoc, H. T., Tran, H. N., & Xuan, T. N. (2024). An improved Artificial Rabbit Optimization for structural damage identification. *Latin American Journal of Solids and Structures*, 21(1), e523.
- Tavakoli, M. M., Shirouyehzad, H., Lotfi, F. H., & Najafi, S. E. (2020). Proposing a novel heuristic algorithm for university course timetabling problem with the quality of courses rendered approach; a case study. *Alexandria Engineering Journal*, 59(5), 3355–3367.
- Algasm, W. A. (2020, November). Hybrid algorithm to solve timetabling problem. In *IOP Conference Series: Materials Science and Engineering* (Vol. 928, No. 3, p. 032053). IOP Publishing.
- Fajrin, A. M., & Fatichah, C. (2020). Multi-parent order crossover mechanism of genetic algorithm for minimizing violation of soft constraint on course timetabling problem. *Register: Jurnal Ilmiah Teknologi Sistem Informasi*, 6(1), 43-51.
- Osorio, J. A., & Esquivel, A. M. (2020). A solution to the university course timetabling problem using a hybrid method based on genetic algorithms. *DYNA: Revista de la Facultad de Minas. Universidad Nacional de Colombia. Sede Medellín*, 87(215), 47-56.
- Qu, X., & Jia, Z. (2025). Multi-strategy enhanced artificial rabbits optimization for prediction of grades in tourism service communication courses. *Scientific Reports*, 15(1), 23854.
- Akkan, C., & Gülcü, A. (2018). A bi-criteria hybrid Genetic Algorithm with robustness objective for the course timetabling problem. *Computers & Operations Research*, 90, 22-32.

- Rappos, E., Thiémarc, E., Robert, S., & Hêche, J. F. (2022). A mixed-integer programming approach for solving university course timetabling problems. *Journal of Scheduling*, 25(4), 391–404.
- Davison, M., Kheiri, A., & Zografos, K. (2024). Modelling and solving the university course timetabling problem with hybrid teaching considerations. *Journal of Scheduling*, 28, 195-215.
- Aldeeb, B. A., Al-Betar, M. A., Norwawi, N. M., Alissa, K. A., Alsmadi, M. K., Hazaymeh, A. A., & Alzaqebah, M. (2021). *Computer and Information Sciences*.
- Ghaffar, A., Din, I. U., Tariq, A., & Zafar, M. H. (2025). Hybridization and artificial intelligence in optimizing university examination timetabling problem: A systematic review. *Review of Education*, 13(2), e70071.
- Kennedy, J., & Eberhart, R. (1995, November). Particle swarm optimization (PSO). In *Proc. IEEE international conference on neural networks*, Perth, Australia (Vol. 4, No. 1, pp. 1942-1948).
- Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
- Goh, S. L., Kendall, G., Sabar, N. R., & Abdullah, S. (2020). An effective hybrid local search approach for the post enrolment course timetabling problem. *Opsearch*, 57(4), 1131-1163.
- Babaei, H., Karimpour, J., & Hadidi, A. (2015). A survey of approaches for university course timetabling problem. *Computers & Industrial Engineering*, 86, 43-59.
- Ceschia, S., Di Gaspero, L., & Schaerf, A. (2023). Educational timetabling: Problems, benchmarks, and state-of-the-art results. *European Journal of Operational Research*, 308(1), 1-18.
- Dimitsas, A., Nastos, V., Gogos, C., & Valouxis, C. (2022, November). An exact based approach for the Post Enrollment Course Timetabling Problem. In *Proceedings of the 26th Pan-Hellenic Conference on Informatics* (pp. 77-82).
- Kaur, M., & Saini, S. (2020). A review of metaheuristic techniques for solving university course timetabling problem. *Advances in Information Communication Technology and Computing: Proceedings of AICTC 2019*, 19-25.

